



TRIBHUVAN UNIVERSITY
INSTITUTE OF SCIENCE AND TECHNOLOGY

The Final Report On
“NEXL: Elevate Your Learning”

Submitted To:
DEPARTMENT OF B.Sc. CSIT
GODAWARI COLLEGE

*In partial fulfillment of the requirements for the degree of Bachelor of Science in Computer
Science and Information Technology (B.Sc. CSIT)*

Submitted By:
SACHIN SHARMA (79011545)
PRASHIKSHA SHRESTHA (79011544)
SIDDHANT DHUNGEL (79011548)

April, 2026

Supervisor Recommendation

I hereby recommend the project entitled “NEXL: Elevate Your Learning” carried out by Sachin Sharma, Prashiksha Shrestha, and Siddhant Dhungel, students of B.Sc. CSIT 2079 Batch at Godawari College, for submission as a partial fulfillment of the requirements for the degree of B.Sc. CSIT.

The project demonstrates the practical application of web technologies in developing an online learning platform that provides users with accessible educational resources and skill-development opportunities. The students have shown satisfactory effort, technical understanding, teamwork, and dedication throughout the development process.

To the best of my knowledge, this work is original and has been completed under my guidance and supervision. Therefore, I recommend this project for evaluation and approval.

.....

Mr. Dipesh Koirala
Supervisor

Certificate Of Approval

The undersigned certify that they have read and recommended to the Department of Computer Science and Information Technology, Godawari College, a project report entitled “NEXL: Elevate Your Learning” submitted by Sachin Sharma, Prashiksha Shrestha, and Siddhant Dhungel. The project was carried out under the supervision and within the time frame prescribed by the syllabus.

We found the students to be hardworking, skilled, and ready to undertake any related work in their field of study and hence recommend the award of partial fulfillment of the Bachelor’s degree of Computer Science and Information Technology.

.....
Supervisor
Dipesh Koirala
Department Of BscCsit
Godawari College

.....
Coordinator
Dipesh Koirala
Department Of BscCsit
Godawari College

.....
External Examiner

.....
Internal Examiner

Acknowledgement

We would like to express our sincere gratitude for the successful development of the “NEXL: Elevate Your Learning” platform. We are especially thankful to the Head of Department, Mr. Om Prakash Dhakal, and our project supervisor, Mr. Dipesh Koirala, for their valuable guidance, continuous support, and encouragement throughout the project. This project provided us with a significant opportunity to apply the theoretical knowledge gained during the B.Sc. CSIT program to the development of a practical system.

We would also like to acknowledge the Department of Computer Science and Information Technology, Godawari College, Itahari, for providing the academic environment, resources, and guidance necessary for completing this project successfully.

Additionally, we are grateful to various online resources, documentation, and learning platforms that supported us during the design and development phases of the project.

Finally, we extend our sincere thanks to everyone who supported us throughout this academic endeavor and contributed to the successful completion of this project.

Abstract

The NEXL: Elevate Your Learning platform is a web-based educational system designed to modernize learning by bridging the gap between traditional methods and digital technology. It provides an interactive and accessible environment where learners can enroll in courses, access structured materials, participate in quizzes, track progress, and earn achievement badges. The system is developed using the MERN stack (MongoDB, Express.js, React.js, Node.js), ensuring efficient data management, responsive interfaces, and secure user authentication.

NEXL integrates automated content generation for course metadata, real-time communication for live classes and chat, and gamification features to enhance student engagement. Additional features include interactive assessments, comprehensive course management, administrative control, e-commerce support, and cross-device responsive design. The platform leverages modern technologies such as Tailwind CSS, Redux Toolkit, Radix UI, Stream SDK, Cloudinary, JWT authentication, and eSewa for local payments.

Currently nearing completion, the system is being refined for optimal performance, responsiveness, and reliability under typical usage conditions. NEXL provides a scalable, user-friendly platform that fosters learner motivation and engagement while offering educators efficient tools for content management and course facilitation.

Keywords: *NEXL, web-based learning, MERN stack, course management, real-time communication, gamification, e-commerce, learner engagement.*

Table of Contents

Supervisor Recommendation	i
Certificate Of Approval	ii
Acknowledgement	iii
Abstract	iv
List of Abbreviations	vii
List of Figures	viii
List of Tables	ix
1 Introduction	1
1.1 Introduction	1
1.2 Problem Statement	2
1.3 Objectives	3
1.4 Scope and Limitations	3
1.5 Development Methodology	4
1.6 Report Organization	6
2 Background Study and Literature Review	8
2.1 Background Study	8
2.2 Literature Review	10
3 System Analysis	13
3.1 System Analysis	13
3.1.1 Requirement Analysis	13
3.1.2 Feasibility Analysis	16

3.1.3	Analysis	19
4	System Design	29
4.1	Design	29
4.2	Algorithm Details	36
5	Implementation and Testing	40
5.1	Implementation	40
5.1.1	Tools Used	40
5.1.2	Implementation Details of Modules	42
5.2	Testing	45
5.2.1	Test Cases for Unit Testing	45
5.2.2	Test Cases for System Testing	47
5.3	Result Analysis	48
6	Conclusion and Future Recommendations	49
6.1	Conclusion	49
6.2	Future Recommendations	49
	REFERENCES	50
	Appendix	52

List of Abbreviations

AI	– Artificial Intelligence
API	– Application Programming Interface
BART	– Bidirectional and Auto-Regressive Transformers
CRUD	– Create, Read, Update, Delete
HTTP	– HyperText Transfer Protocol
HTTPS	– HyperText Transfer Protocol Secure
JWT	– JSON Web Token
LMS	– Learning Management System
MERN	– MongoDB, Express.js, React.js, Node.js
REST	– Representational State Transfer
SDK	– Software Development Kit
SDLC	– Software Development Life Cycle
UML	– Unified Modeling Language
URL	– Uniform Resource Locator
VPS	– Virtual Private Server

List of Figures

1.1	Waterfall Model	4
3.1	Use Case Diagram	14
3.2	Gnatt Chart	19
3.3	Object Diagram	20
3.4	Class Diagram	22
3.5	Sequence Diagram	24
3.6	State Diagram	26
3.7	Activity Diagram	28
4.1	Component Diagram	34
4.2	Deployment Diagram	35
1	Login page	52
2	Main Page	53
3	API TESTING 1	54
4	API TESTING 2	54
5	HMAC-SHA256 signature generation	55
6	Cosine similarity algorithm	55

List of Tables

2.1	Comparison of Existing Solutions vs. NEXL Requirements	12
3.1	Estimated Development Cost of the NEXL Platform	18
4.1	Three-Tier Architecture of NEXL	29
5.1	Unit Test Cases	46
5.2	System Test Cases	47

Chapter 1: Introduction

1.1 Introduction

With the rapid growth of digital technology and internet access, online learning has become an essential part of modern education. E-learning platforms enable students to access educational content anytime using computers, tablets, or smartphones. The global e-learning market has expanded as institutions, instructors, and organizations increasingly adopt digital solutions for courses, assessments, and student interaction. This shift creates learning environments where students actively engage in educational processes through instructional strategies integrated with ICT tools [1].

NEXL: Elevate Your Learning is an online platform connecting students and independent educators in a structured digital environment. Like traditional Learning Management Systems (LMS), NEXL provides tools to manage learning and monitor performance for both students and instructors. Guests can browse courses, view descriptions, and explore categories before registering.

Registered students can enroll in courses, watch video lectures, access notes and PDFs, follow structured lessons, take quizzes, submit assignments, participate in discussions, and rate or review courses. Personalized course recommendations and smart search help learners find relevant content efficiently.

Independent instructors can create and manage courses, upload materials, set pricing, offer previews or discounts, and design quizzes and assignments. Research shows that effective LMS adoption depends on providing tools that meet the specific needs of each user group.

Built with modern web technologies, NEXL delivers a responsive, accessible, and high-performance experience. The platform emphasizes affordability and practical learning, supporting professional development for instructors and reliable access across devices. By empowering both learners and educators, NEXL aims to advance digital education in Nepal.

Furthermore, by integrating intelligent recommendations, smart search capabilities, and interactive features, NEXL enhances the learning experience by helping students find relevant content quickly and efficiently. For educators, the platform provides tools to create and manage courses flexibly, allowing them to reach a wider audience and adapt content to meet learner needs. In this way, NEXL bridges the gap between traditional education and modern digital learning, supporting continuous skill development and professional growth in Nepal.

1.2 Problem Statement

The rapid growth of online education has created new opportunities for flexible and accessible learning worldwide. In Nepal, however, the development of effective and sustainable Learning Management Systems (LMS) is constrained by several practical, financial, and technological challenges. While digital platforms have the potential to transform education delivery, issues related to payment systems, user trust, and platform capabilities continue to hinder their widespread adoption.

Research indicates that despite increasing internet penetration and ongoing digital transformation, the adoption of electronic payment systems in Nepal remains limited due to concerns related to accessibility, usability, and user trust [2]. These challenges directly impact the commercialization and long-term sustainability of online learning platforms.

The major problems that exist in the current market scenario:

- Limited international payment accessibility: Global gateways like Stripe do not support outgoing payments to Nepal, restricting monetization opportunities for local educators [3].
- Digital payment adoption challenges: System unreliability, complex interfaces, usability issues, and low user trust reduce the adoption of online payment platforms [4,5].
- Limited monetization and instructor support features: Many local platforms lack flexible pricing options, analytics tools, and instructor management features necessary for effective course delivery [6].
- Lack of advanced interactive learning capabilities: Existing LMS platforms often lack real-time communication, intelligent personalization, and automated recommendation systems, which significantly lowers student engagement [7].
- Usability and infrastructure limitations: Poor UI design, inconsistent system performance, and technological infrastructure constraints negatively impact the overall user experience and platform adoption [5,7].

These interconnected challenges discourage educators from creating high-quality digital content, reduce student willingness to invest in paid courses, and limit the growth of a sustainable online education ecosystem in Nepal [8]. There is, therefore, a strong need for a modern, locally-tailored LMS that integrates accessible payment solutions, flexible monetization strategies, and intelligent interactive features.

1.3 Objectives

- To build an AI-enhanced e-learning platform using the Waterfall model and implement key learning management features such as course management, user authentication, and progress tracking using the MERN stack.
- To integrate widely used Nepali digital payment systems such as eSewa for secure online transactions.

1.4 Scope and Limitations

Scope

NEXL is a web-based Learning Management System optimized for the Nepali educational landscape. The system encompasses the following core areas:

- **Core Platform:** A responsive MERN-stack application (React, Node.js, Express, MongoDB) designed for reliable performance under varying bandwidth conditions.
- **Payment Integration:** Seamless integration with local digital wallets (eSewa) supporting flexible pricing, coupon mechanisms, and free course previews.
- **Role-Based Management:** Custom workflows for Students (learning, tracking, AI summaries), Instructors (content creation, revenue analytics), and Admins (platform oversight and course approvals).
- **AI-Enhanced Learning:** Automated abstractive summarization of course materials and intelligent metadata generation (tags and descriptions) to streamline content management.
- **Interactive Ecosystem:** Real-time engagement via group chats, 1-on-1 tutoring sessions, and high-quality multimedia delivery including live streaming and PDF resources.

Limitations

- **Platform Restrictions:** Currently limited to a web-based environment; native mobile applications (iOS/Android) are not included.
- **External Dependencies:** AI and multimedia features rely on third-party APIs (Hugging Face, Stream, Cloudinary), making performance subject to their availability and tier limits.

- **Connectivity:** The platform requires an active internet connection; offline mode and local synchronization are not supported.
- **Payment & Security:** Payment processing is restricted to major Nepali gateways. While the system automates quiz evaluations, it does not include advanced biometric proctoring or anti-cheating mechanisms.
- **Discovery Logic:** Course discovery is based on AI-assisted metadata and keywords; a full semantic recommendation engine is reserved for future iterations.

1.5 Development Methodology

The development of the NEXL platform follows a structured Software Development Life Cycle (SDLC) based on the Waterfall Model [9]. The Waterfall approach is a linear and sequential methodology in which each phase must be completed before the next begins. This ensures a well-organized development process with clearly defined stages and deliverables, making it ideal for the academic requirements of this project.

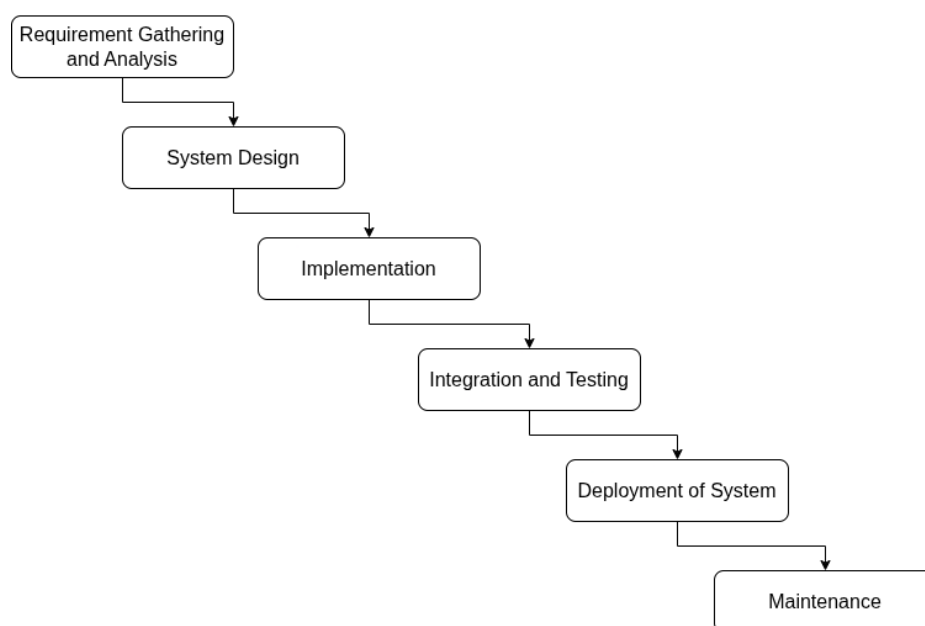


Figure 1.1 Waterfall Model

The sequential phases of the Waterfall model as applied to the development of the NEXL platform are described in detail as follows:

Requirement Gathering and Analysis

This initial phase focused on eliciting and documenting the core functional and non-functional requirements for the NEXL platform. By conducting market analysis, we identified the specific needs of the Nepali educational sector.

- **Key Features Defined:** User registration/login flows, course creation requirements for instructors, student enrollment processes, and local payment integration (eSewa).
- **Deliverable:** A comprehensive Software Requirement Specification (SRS) document outlining all system constraints and mandatory features.

System Design

The technical blueprint of the system was established during this stage, translating requirements into technical specifications.

- **Architecture:** Adoption of the MERN stack (MongoDB, Express, React, Node.js) for a scalable and high-performance system.
- **UML Modeling:** Designing Class Diagrams, Use Case Diagrams, Sequence Diagrams, and State Diagrams to represent the static and dynamic behavior of the system.
- **Database Schema:** Designing the physical schema for MongoDB Atlas, ensuring efficient data relationships and storage for courses, enrollments, and payments.
- **UI/UX Design:** Creating wireframes and mockups for a responsive web interface.

Implementation (Coding)

During the implementation phase, the functional modules were developed based on the design specifications.

- **Frontend Development:** Building the interactive user interface using React.js and Vite, focusing on performance and responsiveness.
- **Backend Development:** Implementing RESTful API endpoints using Node.js and Express to handle business logic, authentication (JWT), and database operations.
- **Service Integration:** Integrating third-party services such as Cloudinary for media management, the Hugging Face API for AI-assisted content generation, and the eSewa SDK for payment processing.

Testing and Validation

Once the modules were developed, they were integrated and subjected to rigorous verification to ensure quality and reliability.

- **Unit Testing:** Verifying the functionality of individual components and API controllers.
- **Integration Testing:** Ensuring seamless communication between the React frontend and the Express backend.

- **System Testing:** Validating the end-to-end user journeys, such as searching for a course, making a payment via eSewa, and accessing course materials.
- **AI Accuracy Testing:** Evaluating the relevance and quality of the AI-generated course summaries and tags.

Deployment

The finalized platform was released to a production environment to ensure public accessibility and reliability.

- **Cloud Hosting:** Deploying the frontend on Vercel and the backend services on Render for high availability.
- **Environment Configuration:** Setting up production-ready environment variables, SSL certificates, and database connections.
- **CI/CD Pipeline:** Establishing automated deployment workflows to facilitate efficient system updates.

Maintenance

Post-deployment, the platform entered the maintenance phase to ensure long-term sustainability.

- **Monitoring:** Tracking system performance and uptime using cloud logs.
- **Bug Fixing:** Addressing issues identified through initial user testing and feedback.
- **Feature Roadmap:** Identifying potential areas for future enhancement, such as native mobile applications and advanced recommendation algorithms.

1.6 Report Organization

This project report is organized into six chapters to document the complete development lifecycle of the NEXL platform:

Chapter 1: Introduction provides the background of the project, problem statement, objectives, scope and limitations, development methodology, and report organization.

Chapter 2: Background Study and Literature Review discusses fundamental theories related to Learning Management Systems, e-learning in developing countries, artificial intelligence in education, and reviews existing global and local platforms with gap analysis.

Chapter 3: System Analysis presents requirement analysis (functional and non-functional), feasibility analysis (technical, operational, economic, and schedule), and object-oriented analysis using UML diagrams including use case, class, sequence, state, and activity diagrams.

Chapter 4: System Design describes the system architecture, database design, interface design, and detailed algorithms including course recommendation, content summarization, and search algorithms.

Chapter 5: Implementation and Testing covers the technological stack, implementation details of modules, unit testing, system testing, and result analysis.

Chapter 6: Conclusion and Future Recommendations summarizes the work completed, achievements, and suggests future enhancements for the platform.

The report also includes references, bibliography, and appendices containing screenshots, source code snippets, and supervisor visit logs.

Chapter 2: Background Study and Literature Review

2.1 Background Study

i. Learning Management Systems (LMS):

A Learning Management System (LMS) is a web-based platform designed to deliver, manage, and evaluate educational content in a structured manner [1]. It acts as a centralized system where instructors can upload materials, track student progress, and conduct assessments. LMS platforms are widely used in schools, universities, and corporate training environments due to their flexibility and accessibility. They support both synchronous and asynchronous learning, making education more accessible regardless of location. Key functionalities include content delivery, user management, assessment tools, communication features, and analytics for performance tracking. Modern LMS platforms are built using scalable architectures such as service-oriented design, RESTful APIs, and microservices, enabling seamless integration with third-party services. These systems also support mobile learning, ensuring accessibility on smartphones and tablets.

ii. E-Learning in Developing Countries :

E-learning has emerged as a powerful solution for expanding access to education in developing countries. However, several challenges limit its effectiveness, including poor internet infrastructure, low digital literacy, and financial constraints. In countries like Nepal, reliable internet access is not always available, especially in rural areas, making it difficult to use high-bandwidth platforms. Therefore, e-learning systems must be optimized for low bandwidth and mobile devices. Additionally, users often rely on smartphones rather than computers, which influences system design. Financial limitations also affect the adoption of paid platforms, emphasizing the need for affordable solutions. During the COVID-19 pandemic, the rapid shift to online learning exposed weaknesses such as lack of localization, limited interactivity, and absence of integrated payment systems [8]. These challenges highlight the importance of developing context-aware LMS platforms tailored to local needs.

iii. Artificial Intelligence in Education :

Artificial Intelligence (AI) is transforming modern education by improving efficiency, personalization, and learner engagement. AI-powered systems can analyze learner behavior and adapt content based on individual learning patterns, enabling personalized learning experiences where students receive recommendations suited to their progress and interests. These adaptive systems are shown to tailor instruction to individual student needs and promote

better engagement and performance. AI also automates repetitive tasks such as grading and content organization, reducing the workload for instructors and allowing educators to focus more on teaching and creative instructional tasks. In addition, intelligent tutoring systems can provide real-time feedback and guidance, emulating one-on-one tutoring and enhancing the learning process. The integration of AI into Learning Management Systems (LMS) enhances both teaching and learning processes, making AI-driven education systems increasingly popular in academic and professional environments [10].

iv. Automatic Text Summarization :

Automatic text summarization is an AI technique used to generate concise summaries from large volumes of text. Modern approaches use transformer-based models such as BERT and BART, which rely on encoder-decoder architectures and attention mechanisms. These models understand context and semantics, producing accurate and meaningful summaries. Summarization is particularly useful in e-learning, where students need to quickly grasp key concepts from lengthy materials. It reduces study time while improving comprehension and retention. This feature is especially beneficial for mobile users who prefer shorter, digestible content [11].

v. Recommendation Systems :

Recommendation systems enhance user experience by suggesting relevant learning materials based on user preferences and behavior. Content-based filtering techniques use embeddings such as E5 and MPNet to represent textual data in vector form. Similarity measures like cosine similarity are then used to match users with appropriate content. Advanced systems utilize vector databases such as Faiss or Pinecone for efficient storage and retrieval of embeddings. These systems improve engagement by guiding learners toward relevant courses and materials. As a result, recommendation engines play a crucial role in modern AI-powered LMS platforms [12].

vi. Digital Payment Integration in E-Education :

Digital payment integration is essential for enabling monetization in online education platforms. It allows instructors to sell courses and users to make secure transactions. In Nepal, local payment gateways such as eSewa provide affordable transaction options compared to international services like PayPal or Stripe. These local solutions typically charge lower transaction fees, making them suitable for small-scale platforms [2]. Integration involves APIs for payment initiation, verification, and status tracking. Security is a critical aspect, requiring encryption protocols such as SSL/TLS and mechanisms like HMAC for data integrity. Proper transaction handling ensures reliability and user trust in the system.

vii. Online Education in Nepal:

In Nepal, online education is still developing and largely depends on general-purpose tools such as Zoom, Google Meet, and social media platforms. During the pandemic, approximately 68% of institutions used platforms like Facebook and YouTube for teaching. These tools lack structured LMS capabilities such as assessment management and progress tracking. Local platforms such as Mero School and Gurukul primarily focus on K-12 education and offer limited flexibility. They also lack monetization features and AI-based enhancements, which are essential for modern e-learning systems [8].

2.2 Literature Review

Global Learning Management System

i. Moodle:

Moodle is one of the most widely used open-source LMS platforms, known for its flexibility and extensive feature set. It supports course management, assessments, and collaborative learning tools. Being open-source, it allows customization according to institutional needs. However, its setup and maintenance require technical expertise, which can be a barrier in resource-limited environments. Additionally, performance optimization and scalability can be challenging without proper infrastructure [13].

ii. Canvas and Blackboard:

Canvas and Blackboard are commercial LMS platforms offering advanced features such as analytics, integrations, and cloud hosting. They provide a user-friendly interface and robust support services. However, these platforms are expensive and often not affordable for institutions in developing countries. They also lack integration with local payment systems, limiting their usability in regions like Nepal [14].

iii. Google Classroom:

Google Classroom is a free and easy-to-use platform widely adopted during the COVID-19 pandemic. It integrates well with other Google services and supports basic classroom management features. However, it lacks advanced functionalities such as monetization, deep customization, and AI-driven analytics. This makes it less suitable for scalable and feature-rich educational platforms [15].

iv. Coursera:

Coursera is a global online learning platform offering structured courses, professional

certificates, and degree programs in collaboration with universities and companies. Its academic partnerships and structured course design help improve learner engagement and completion rates. Coursera is ideal for students seeking recognized certifications but may be less flexible for self-paced or highly practical learning needs [16].

v. Udemy:

Udemy is an online learning marketplace where instructors can create courses on a wide variety of topics. It emphasizes practical skill acquisition, flexibility, and affordable self-paced learning. However, the quality of courses may vary due to its open platform nature, and it lacks formal academic recognition compared to university-backed platforms like Coursera [17].

Local Learning Management Systems

vi. MySecondTeacher:

mySecondTeacher is a Nepali e-learning platform designed to support school-level education with curriculum-aligned interactive video lessons, diagnostic assessments, and performance tracking. It bridges gaps in traditional classroom instruction by offering interactive digital resources for students, teachers, and parents. Its localized content makes it highly relevant for the Nepali educational context [18].

Gap Analysis:

Existing systems show several limitations when applied to the Nepali context. International LMS platforms lack local payment integration and are often expensive, while local platforms lack advanced features such as AI and scalability. There is a clear gap between available solutions and the requirements of a modern, localized LMS. This highlights the need for a system like NEXL, which integrates AI features, supports local payments, and provides flexible monetization options tailored to Nepal.

Table 2.1 Comparison of Existing Solutions vs. NEXL Requirements

Feature	International LMS [1, 19]	Local Platforms [18]	NEXL (Proposed)
Local Payment Integration	Not Available	Partial	Full (eSewa)
AI Summarization	Premium/Add-on	Not Available	Integrated
Recommendation Engine	Basic	Not Available	Vector-based
Instructor Monetization	High Commission	Fixed Pricing	Flexible
Real-time Interaction	Limited	Basic	Integrated

Chapter 3: System Analysis

3.1 System Analysis

3.1.1 Requirement Analysis

I.Functional Requirements

The functional requirements define the specific behaviors and functions that the NEXL platform must perform to meet the needs of students, instructors, and administrators. These requirements are categorized based on user roles and system modules.

User Management Module:

- **Authentication:** Secure user registration and login with JWT token-based authentication, supporting email/password and OAuth (Google/GitHub) providers.
- **Profile Management:** Users can create and update profiles, upload profile pictures, and manage notification preferences.
- **Role-Based Access Control:** Three distinct roles (Student, Instructor, Admin) with specific permissions enforced through middleware.

Course Management Module:

- **Course Creation:** Instructors can create courses with titles, descriptions, syllabi, and pricing in NPR (Nepali Rupees).
- **Content Upload:** Support for video files (MP4), documents (PDF), and images with Cloudinary integration.
- **Course Catalog:** Students can browse, search, and filter courses by category, price, and rating.
- **Enrollment Management:** Automated enrollment upon payment verification with access control to course materials.

Payment and Monetization Module:

- **Local Payment Integration:** Seamless integration with eSewa APIs for NPR transactions.

- **Coupon System:** Instructors can generate discount codes with percentage or fixed amount reductions.
- **Revenue Tracking:** Dashboard for instructors to view earnings, enrollments, and transaction history.

Interactive Features:

- **Real-time Communication:** Stream Video SDK-based group chat and one-on-one messaging between students and instructors.
- **Assessment System:** Quiz creation with multiple-choice questions, automated grading, and instant feedback.
- **Gamification:** Digital badges awarded for course completion, high quiz scores, and participation.

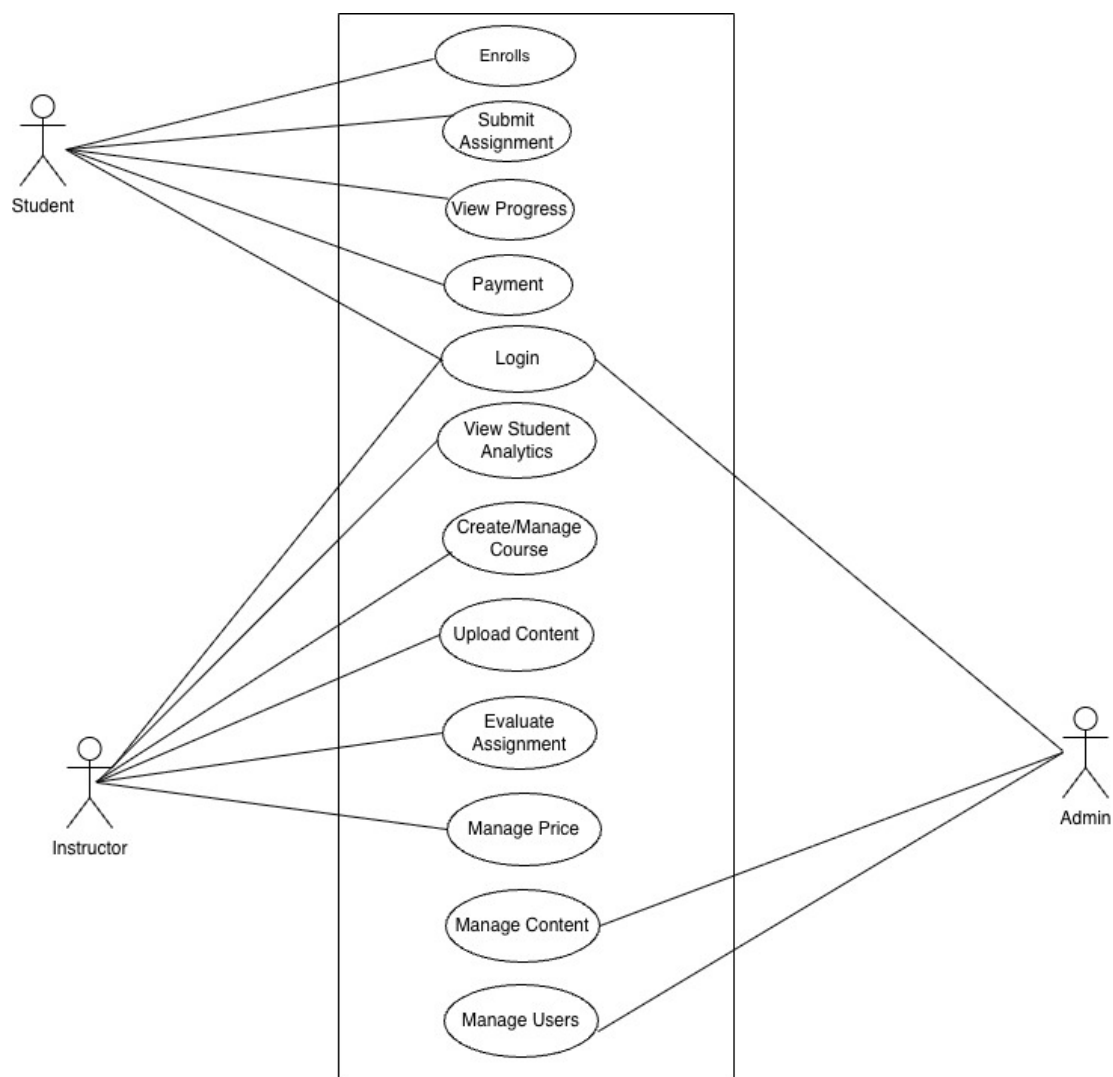


Figure 3.1 Use Case Diagram

Use Case Description: The use case diagram in Figure 3.1 illustrates the primary actors and their interactions with the NEXL system:

- Student: Can browse courses, enroll (free or paid), view content, take quizzes, submit assignments, participate in discussions, and receive AI-generated summaries.
- Instructor: Can create/manage courses, upload multimedia content, set pricing and coupons, evaluate assignments, view analytics, and communicate with students.
- Admin: Manages user accounts, approves published courses, monitors system health, and handles payment disputes.

ii.Non Functional Requirements

Non-functional requirements specify the quality attributes, performance standards, and constraints under which the system must operate.

Performance Requirements:

- The system shall load the landing page within 3 seconds on a 5 Mbps internet connection (standard Nepali broadband).
- Video streaming shall support adaptive bitrate switching (480p, 720p, 1080p) to accommodate varying bandwidth conditions.
- The recommendation engine shall return results within 500ms for a catalog of up to 1,000 courses.

Security Requirements:

- Authentication: JWT tokens with 24-hour expiration and refresh token rotation.
- Data Protection: HTTPS/TLS 1.3 encryption for all data transmission; bcrypt hashing for passwords with salt rounds ≥ 10 .
- Payment Security: HMAC-SHA256 signature verification for payment gateway webhooks; no storage of sensitive banking credentials.
- Input Validation: XSS and SQL injection prevention through parameterized queries and input sanitization.

Usability and Accessibility:

- Responsive design supporting viewports from 320px (mobile) to 1920px (desktop).

- WCAG 2.1 Level AA compliance for accessibility (alt text for images, keyboard navigation, ARIA labels).
- Intuitive navigation with maximum 3-click depth to reach any course content.

Scalability and Reliability:

- Horizontal scaling capability through stateless REST API architecture.
- 99.5% uptime SLA with automated database backups every 24 hours.
- Support for concurrent users: minimum 500 simultaneous video streams.

3.1.2 Feasibility Analysis

I. Technical Feasibility

The NEXL platform is technically feasible and available open-source technologies.

Technology Stack Viability:

- Frontend: React.js 18+ with Vite build tool provides component-based architecture and efficient hot module replacement for development.
- Backend: Node.js with Express.js offers non-blocking I/O suitable for real-time chat and concurrent video streaming.
- Database: MongoDB Atlas provides flexible schema design for varying course structures and automatic sharding for scalability.
- AI Integration: Hugging Face Inference API with DistilBART (400 MB) models is feasible on a standard render free tier hosting
- Payment APIs: eSewa provides well-documented REST APIs with SDKs for Node.js, supporting sandbox environments for testing.

Infrastructure Requirements:

- Development: Standard laptop with 8 GB RAM, Node.js runtime, and MongoDB Community Edition.
- Production: React on Vercel And Node and Express on Render.
- Media Storage: Cloudinary free tier (25 GB) is sufficient for initial deployment.

Operational Feasibility

Operational feasibility assesses whether the system aligns with existing organizational workflows and user capabilities.

User Acceptance:

- Students: Familiarity with e-learning platforms post-COVID-19; smartphone-first design accommodates primary device usage in Nepal.
- Instructors: Minimal technical training required due to intuitive dashboard design similar to YouTube Studio or Google Classroom.
- Administrators: Web-based admin panel with role-based access eliminates the need for specialized software installation.

Organizational Integration:

- The platform operates independently of existing institutional systems, requiring no integration with legacy university ERP systems.
- Content migration support: instructors can bulk-upload PDFs and MP4 files via a drag-and-drop interface.
- Support for both synchronous (live streaming) and asynchronous (recorded) learning modes accommodates varying internet reliability.

Maintenance Plan:

- Content Moderation: Admin approval workflow for courses before public visibility ensures quality control.
- Update Cycle: Monthly updates for security patches; quarterly feature updates based on user feedback.
- Support System: Built-in ticketing system for technical issues with a 48-hour SLA for critical payment-related bugs.

II. Economic Feasibility

Economic feasibility demonstrates that the project is financially viable with minimal upfront investment and sustainable operational costs.

Development Costs (One-time):

- Software Licenses: \$0 — Open-source stack (React, Node.js, MongoDB, Express).

- Development Hardware: \$0 — Existing personal computers and laptops.
- AI API: \$0 — Hugging Face Inference API free tier.
- Internet/Utilities: \$0 (development phase only).

Deployment Costs (Monthly Recurring):

- Database: \$0 — MongoDB Atlas M0 free tier sufficient for initial 1,000 users.
- Media Storage: \$0/month (Cloudinary free tier initially).
- Domain & SSL: \$0/year (.com.np domain + Let's Encrypt SSL).

Revenue Model:

- Commission Structure: 5–10% platform fee per transaction.
- Break-even Analysis: With 50 courses sold monthly at an average of NPR 1,000, the platform generates NPR 50,000 revenue; a platform fee of NPR 2,500–5,000 covers hosting costs.
- Teacher Incentive: Higher profit margins (90–95%) encourage content Creation.

Cost-Benefit Analysis: Total initial investment: NPR 0. With a projected monthly transaction

Table 3.1 Estimated Development Cost of the NEXL Platform

Cost Component	Amount (NPR)	Remarks
Software & Development Tools	0	Open-source (Node.js, React, MongoDB)
Hardware	0	Personal PC
AI Inference API	0	Hugging Face Free-Tier
Internet & Connectivity	0	Development Phase (Pre-production)
Vercel and Render Hosting	0	DigitalOcean / Render (Trial/Free Tier)
Maintenance	0	Periodic student-led updates
Total Cost	0	Economically viable for local adoption

System Architecture Overview

Schedule

The project timeline spans 16 weeks (one academic semester), managed through a sequential Waterfall model consisting of distinct phases: Requirements, Design, Implementation, Testing, and Deployment.

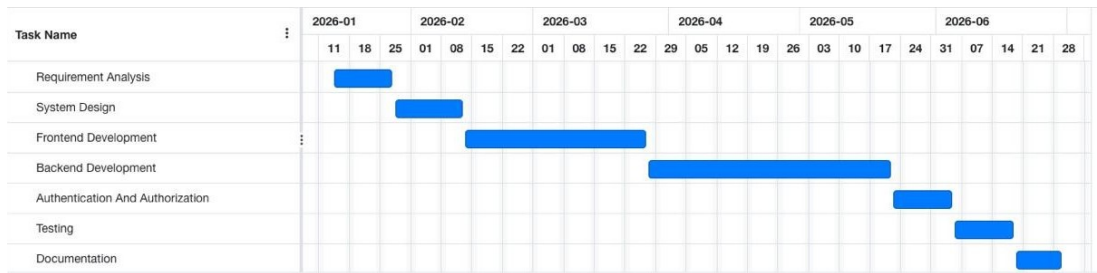


Figure 3.2 Gantt Chart

Schedule Description:

The roadmap presented in Figure 3.2 illustrates the 16-week project schedule for the NEXL platform. The project followed a structured Waterfall approach to ensure that each phase was completed and verified before moving to the next.

The development journey began with the Requirement Analysis phase, which established the core functional and non-functional specifications. This was followed by the System Design phase, defining the architectural blueprint. The Implementation phase involved the sequential development of the frontend and backend modules, ensuring that the system was built upon a solid design foundation.

The later stages of the schedule focused on Testing, where the integrated system was verified against all requirements, followed by Deployment and Documentation. This linear progression provided a clear and manageable path for the development of the NEXL platform, ensuring that all project deliverables were met within the academic semester.

3.1.3 Analysis

The system analysis follows the Object Oriented Approach utilizing UML (Unified Modeling Language) diagrams to model the static structure, dynamic behavior, and business processes of the NEXL platform. Object modelling using Class and Object Diagrams Object modeling represents the instances of classes and their relationships at a specific point in time, illustrating the runtime state of the system. Object Diagram (Instance Level): An object diagram for a specific enrollment scenario is shown in Figure 3.3, demonstrating the runtime state when a student accesses a course:

- sachin:Student with attribute values (name="Sachin Sharma", email="sachin@email.com", enrolledCourses=[webDev, dataScience], badges=[fastLearner])
- prashiksha:Instructor with attribute values (name="Prashiksha Shrestha", coursesCreated=[webDev], revenue=45000)

- webDev:Course with state (title="Complete Web Development", price=1500, discountPrice=1200, status=PUBLISHED, category="Technology")
- enroll1:Enrollment linking sachin and webDev with (enrolledDate="2025-03-15", paymentStatus="COMPLETED", progressPercentage=45%)
- payment1:Payment with (amount=1200, gateway="eSewa", transactionId="TXN123456", status="VERIFIED")

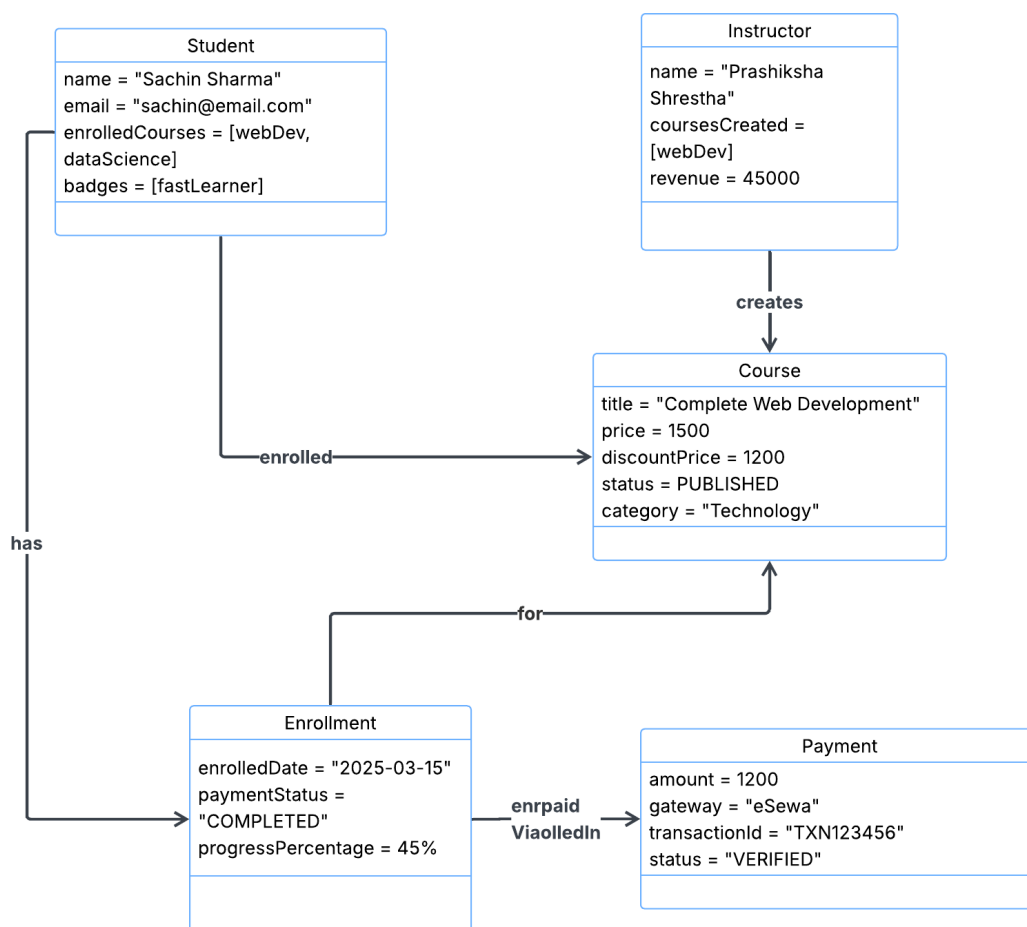


Figure 3.3 Object Diagram

This demonstrates the runtime state of the system when a specific student accesses a course, including the relationships between objects and their current attribute values. Class Diagrams

Class modeling represents the static structure of the system, showing classes, their attributes, methods, and relationships. The class diagram in Figure 3.4 illustrates the main entities of the NEXL platform and their relationships:

- User: Abstract base class with attributes (+String name, +String role, +String email).

Serves as parent class for Instructor, Student, and Admin with 1-to-1 inheritance relationships.

- Instructor: Specialized User who creates and manages courses. Maintains 1-to-many relationship with Course (Instructor creates multiple courses).
- Student: Specialized User who enrolls in courses. Maintains 1-to-many relationship with Enrollment (Student can have multiple enrollments).
- Admin: Specialized User with platform-level permissions. Maintains 1-to-many relationship with Course (Admin approves/moderates multiple courses) and User (Admin manages user accounts).
- Course: Core entity with attributes (+String title, +String category, +Number price, +String status). Contains 1-to-many Sections (aggregation relationship). Associated with Instructor who created it and Admin who approved it.
- Section: Course component with attributes (+String title, +Number order). Contains 1-to-many Lessons (aggregation relationship). Belongs to exactly one Course.
- Lesson: Learning unit with attribute (+String title). Contains 1-to-many Content items and evaluates through 1-to-many Quizzes. Belongs to exactly one Section.
- Content: Learning material with attributes (+String title, +String type). Part of Lesson composition (video, PDF, etc.).
- Quiz: Assessment component with attributes (+String title, +Number timeLimit). Associated with Lesson for student evaluation.
- Enrollment: Association class linking Student to Course with attributes (+Number progress, +String status). Represents the enrollment record with progress tracking.
- Badge: Achievement entity with attributes (+String name, +String type). Earned by Student through Enrollment (1-to-1 relationship with Enrollment via "Processed_By").
- Payment: Transaction entity with attributes (+Number amount, +String gateway, +String status). Earned through Enrollment (1-to-many relationship via "Earns" - one enrollment generates one payment).

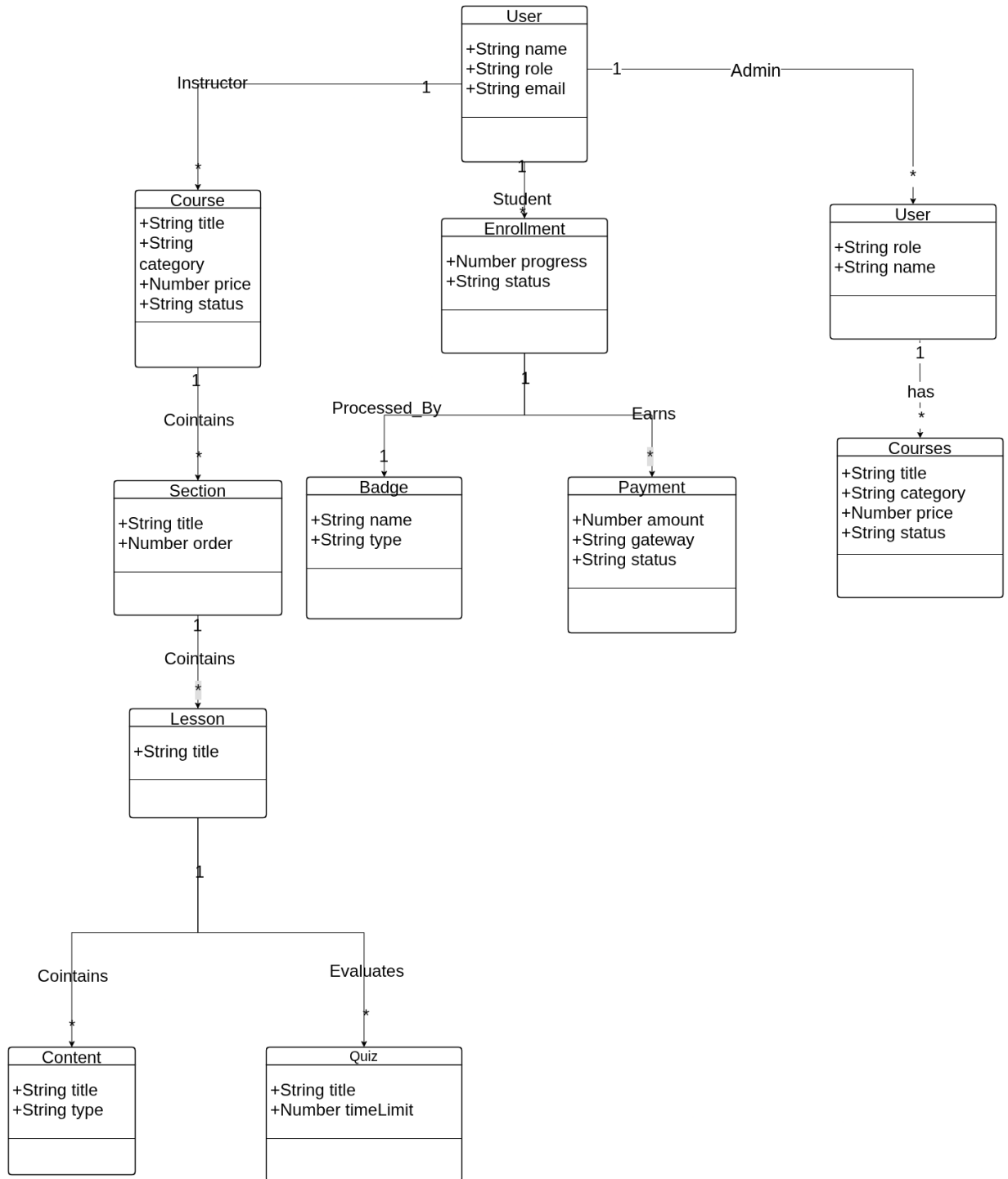


Figure 3.4 Class Diagram

Key Relationships:

- Inheritance: Instructor $\leftarrow$ User, Student $\leftarrow$ User, Admin $\leftarrow$ User (solid line with 1-to-1 multiplicity)
- Association:
 - Instructor $\rightarrow$ Course (1-to-many, creates relationship)
 - Admin $\rightarrow$ Course (1-to-many, approves/manages relationship)
 - Student $\rightarrow$ Enrollment (1-to-many, enrolls relationship)
 - Enrollment $\rightarrow$ Badge (1-to-1, processed_by relationship)
 - Enrollment $\rightarrow$ Payment (1-to-1, earns relationship)
- Aggregation (Contains):
 - Course $\diamond$ Section (1-to-many)
 - Section $\diamond$ Lesson (1-to-many)
 - Lesson $\diamond$ Content (1-to-many)
- Association: Lesson $\rightarrow$ Quiz (1-to-many, evaluates relationship)

Dynamic modelling using State and Sequence Diagrams Dynamic modeling illustrates the time-dependent behavior of objects and their interactions through message passing over time.

Sequence Diagram: Course Enrollment with Payment The sequence diagram in Figure 3.5 illustrates the interaction flow for a student enrolling in a paid course:

1. Actor: Student initiates enrollment via Frontend.
2. Step 1: Frontend sends POST /enroll request to Backend API with courseId and userId.
3. Step 2: Backend validates JWT token and checks if course exists and is published.
4. Step 3: Backend checks Enrollment collection to prevent duplicate enrollment.
5. Step 4: If paid course, Backend initiates payment with selected Gateway (eSewa).
6. Step 5: Payment Gateway returns redirect URL; Frontend redirects user to payment page.
7. Step 6: User completes payment on Gateway; Gateway sends webhook to Backend.
8. Step 7: Backend verifies payment signature and updates Enrollment status to "COMPLETED".

9. Step 8: Backend grants access to Course Content; Frontend displays success message.

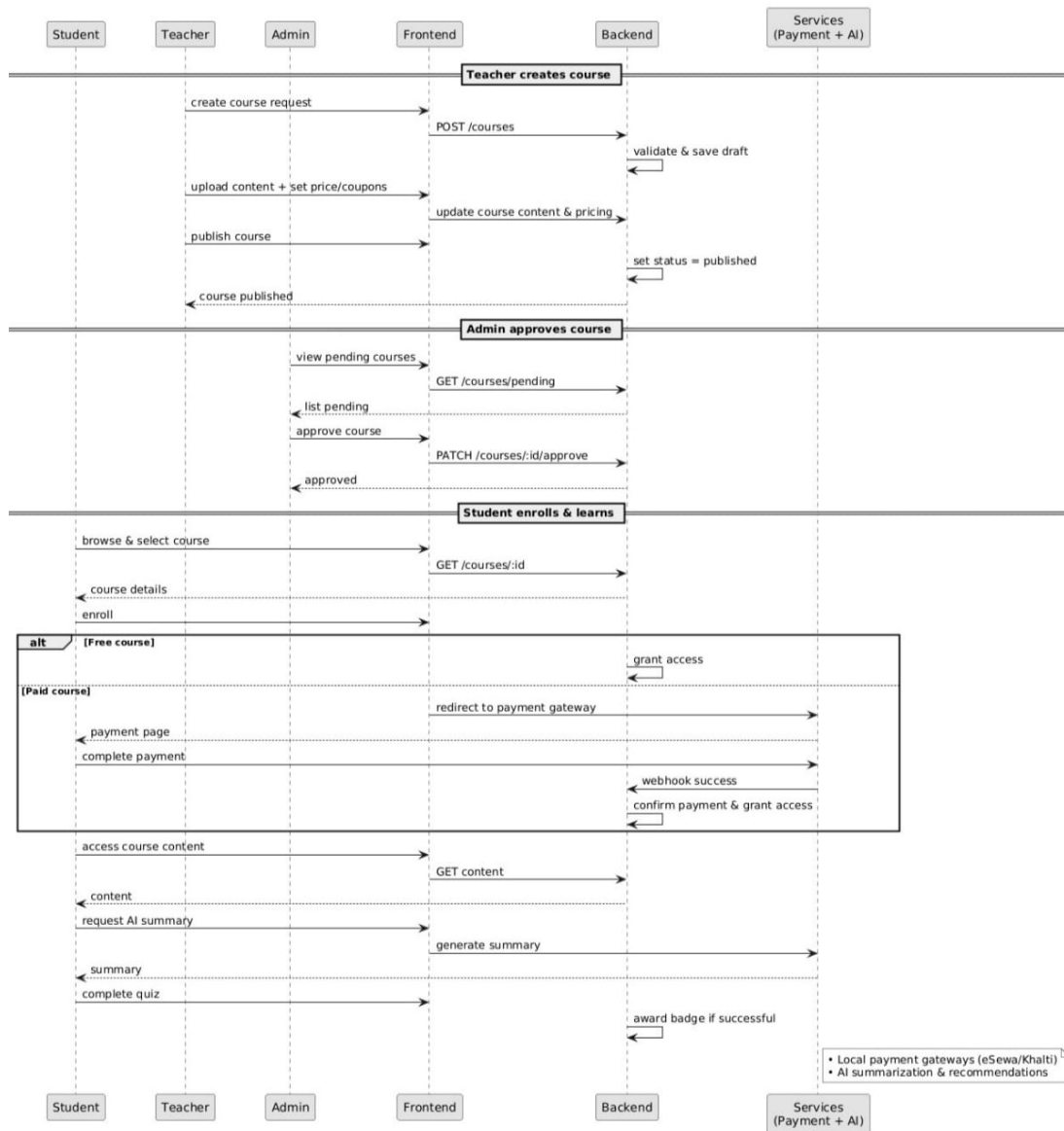


Figure 3.5 Sequence Diagram

Dynamic Modelling using State Diagrams

State diagrams illustrate the lifecycle states and transitions of key objects in response to events. The state diagram in Figure 3.6 shows the state transitions for a Course object from creation to archival:

- **Initial State:** Course created by Instructor enters DRAFT state; content editable but not visible to students.
- **Transition 1:** Instructor submits course for review via `submitForReview()` event; Course transitions to PENDING_APPROVAL state.

- Transition 2: Admin reviews course and approves via approve() event; Course transitions to PUBLISHED state and becomes visible in catalog.
- Transition 3: Admin rejects course via reject() event; Course transitions to REJECTED state and returns to Instructor with feedback.
- Transition 4: Instructor revises rejected course via revise() event; Course returns to DRAFT state for editing.
- Transition 5: Instructor or Admin archives published course via archive() event; Course transitions to ARCHIVED state.
- Transition 6: Instructor reactivates archived course via reactivate() event; Course returns to PENDING_APPROVAL state for re-review.
- Final State: ARCHIVED or PUBLISHED (with possible transitions between them).

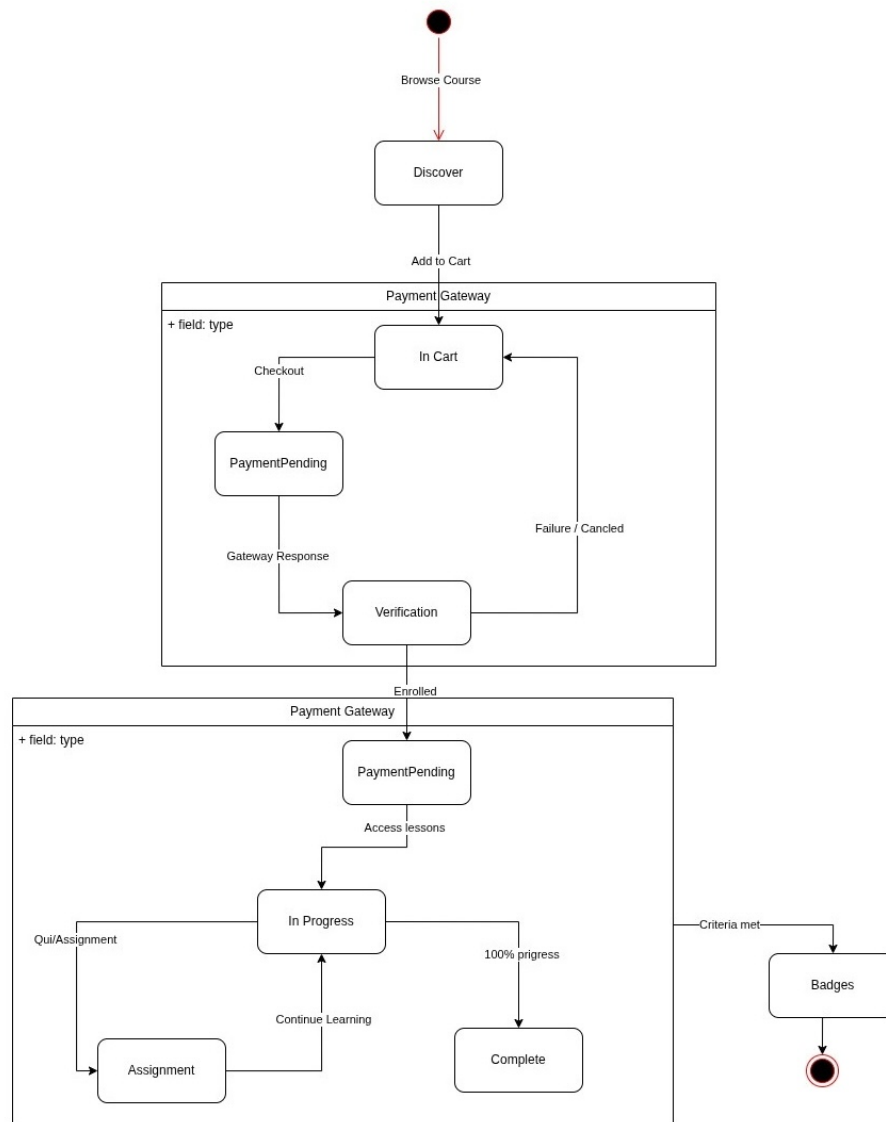


Figure 3.6 State Diagram

State Descriptions:

- **DRAFT:** Initial creation state; Instructor can edit all course content, pricing, and structure; not visible to students.
- **PENDING_APPROVAL:** Awaiting Admin review; content locked from editing; Admin evaluates quality and policy compliance.
- **PUBLISHED:** Live state; visible in course catalog; students can browse, enroll, and access content; Instructor can monitor analytics.
- **REJECTED:** Failed review state; contains Admin feedback; Instructor must address issues before resubmission.

- **ARCHIVED:** Retired state; existing enrolled students retain access; new enrollments disabled; hidden from catalog.

Process Modelling using Activity Diagrams

Process modeling using Activity Diagrams illustrates the workflow and business logic of the course enrollment process, including decision points and parallel activities.

Activity Diagram: Course Enrollment and Payment Process

The activity diagram in Figure 3.7 models the complete workflow from browsing to content access:

Swimlanes (Partitions):

1. Student: Actions performed by the learner.
2. System: Backend processing and validation.
3. Payment Gateway: External eSewa/ services.
4. Instructor: Notifications and revenue updates.

Flow Description:

1. Initial Node: Student browses course catalog.
2. Decision: Is user authenticated?
 No → Redirect to Login Activity → Return to Browse.
 Yes → View Course Details.
3. Decision: Is course free?
 Yes → Direct Enrollment Activity → Grant Access.
 No → Check/Apply Coupon Activity → Calculate Final Price.
4. Fork (Parallel):
 - Path A: Redirect to Payment Gateway (eSewa/).
 - Path B: Create Pending Enrollment Record in Database.
5. Decision: Payment Success?
 Success → Join → Update Enrollment Status → Notify Instructor → Grant Course Access.

Failure → Retry Payment (max 3 attempts) → [Abort] → Cancel Enrollment.

6. Final Node: Student accesses learning content (videos, quizzes).

Key Decision Nodes:

- [Course Capacity Full?]: Checks maxStudents limit before allowing enrollment.
- [Valid Coupon?]: Validates coupon code and expiry date.
- [Payment Verified?]: HMAC signature verification from gateway webhook.

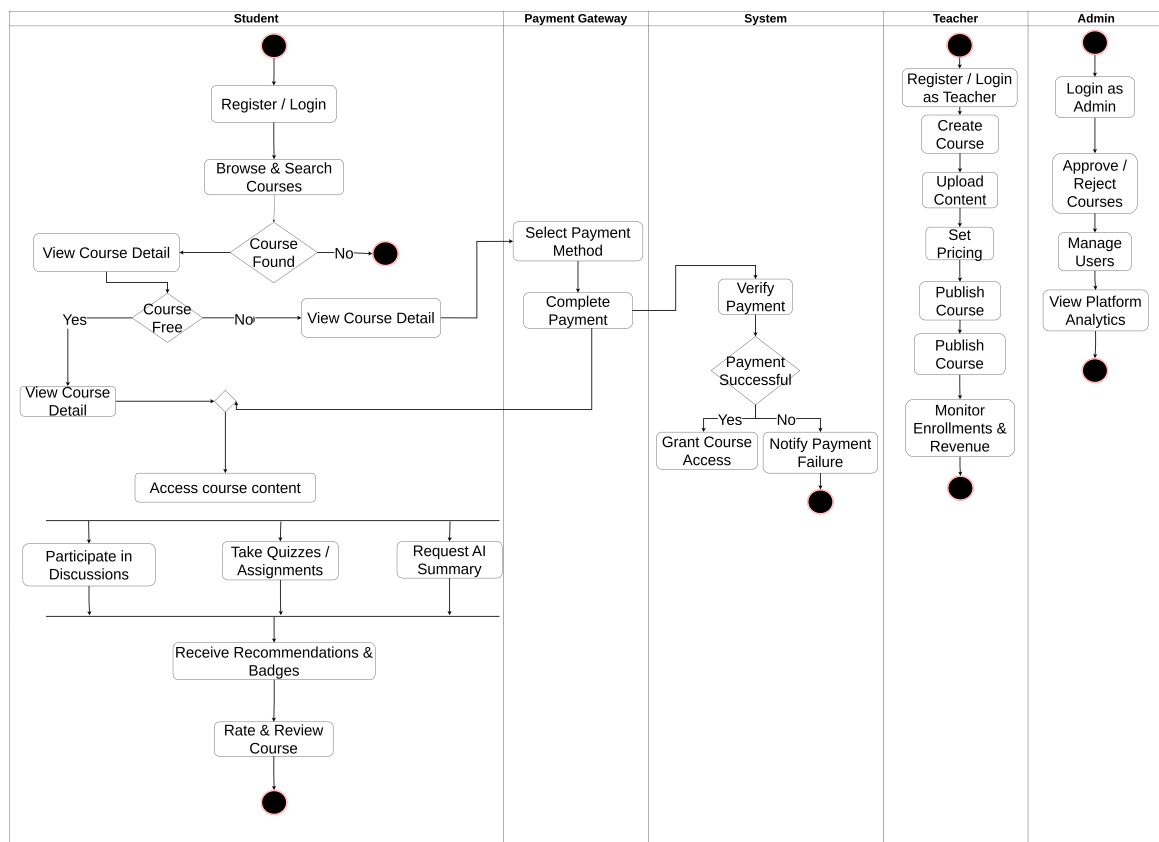


Figure 3.7 Activity Diagram

Process Optimization: The diagram reveals that the Apply Coupon activity can be optimized by caching valid coupon codes in Redis to reduce database queries. The Verify Payment activity includes a timeout guard condition to handle gateway latency, ensuring the system remains responsive even if the external payment service is slow.

Chapter 4: System Design

This chapter presents the complete design of the NEXL platform, a full-stack Learning Management System built with React, Node.js, and MongoDB. The design follows an Object-Oriented approach consistent with the analysis phase, and covers the architectural overview, refined UML diagrams, component and deployment design, and the core algorithms that power the platform.

4.1 Design

The NEXL platform follows an object-oriented design approach using UML modeling techniques and is implemented using the MERN stack (MongoDB, Express.js, React.js, Node.js). The system is designed using a layered architecture that separates the frontend, backend, database, and external services. This ensures modularity, scalability, and maintainability of the system. Table 4.1 summarises the three tiers.

Table 4.1 Three-Tier Architecture of NEXL

Tier	Technology	Responsibility
Presentation (Frontend)	React 18 + Vite	Student, instructor, and admin user interfaces
Application (Backend)	Node.js + Express	Business rules, REST API, and AI services
Data (Database)	MongoDB + Mongoose	Persistence of all domain objects

Refinement of UML Design

The UML diagrams presented in Chapter 3 form the basis of the system design. In this chapter, they are interpreted from an implementation perspective to guide system development.

i. Class Diagram

The Class Diagram presented in Figure 3.4 provides a structural blueprint of the NEXL system, capturing the key domain entities, their attributes, methods, and the relationships that bind them together. Core Entities and Inheritance: At the root of the hierarchy sits the User class, which encapsulates the attributes common to all system participants: id, name, email, password (hashed), role (student, instructor, or admin), avatar, isEmailVerified, and createdAt. Three specialised subclasses inherit from User via generalisation:

- Student extends User with enrolledCourses, badges, totalXP, and streakDays to support gamification.

- Instructor extends User with bio, expertise, createdCourses, and earnings.
- Admin extends User with permissions and methods such as approveCourse and manageUsers.

Course and Content Structure: The Course class holds title, description, category, level, price, status (draft, pendingReview, published, or archived), thumbnail, embedding (a 384-dimensional vector), enrollmentCount, and instructorId. A Course is composed of many Lecture objects, each storing title, videoUrl, duration, summary (AI-generated), and isFree. Each Lecture may be associated with a Quiz, which in turn aggregates multiple Question objects, each carrying questionText, options, correctAnswer, and an aiGenerated flag.

Enrollment and Payment: The Enrollment class acts as an association class between Student and Course, recording enrolledAt, completedLectures, progressPercent, and certificateIssued. Every Enrollment is linked to exactly one Payment record, which stores transactionUUID, amount, productCode, status (initiated, pending, complete, or failed), esewaSignature, and verifiedAt.

Supporting Classes: The Submission class records student quiz attempts, holding studentId, quizId, answers, score, and submittedAt. The Badge class models gamification rewards with name, icon, xpRequired, and awardedAt. The LiveClass class manages real-time sessions with streamCallId, instructorId, courseId, scheduledAt, and status.

ii. Object Diagram

The Object Diagram in Figure 3.3 captures a concrete runtime snapshot of the NEXL system, illustrating how class definitions are instantiated into specific objects with real attribute values during a typical student enrollment scenario. **Instantiated Objects:** The diagram presents the following runtime instances:

- student1 of type Student has id equal to stu_001, name equal to Aarav Sharma, email equal to aarav@gmail.com, role equal to student, totalXP equal to 320, and streakDays equal to 5.
- instructor1 of type Instructor has id equal to ins_001, name equal to Priya Thapa, email equal to priya@nexl.com, role equal to instructor, and expertise equal to Web Development.
- course1 of type Course has id equal to crs_001, title equal to Full-Stack Web Development with MERN, category equal to Programming, level equal to Intermediate, price equal to 1499, status equal to published, instructorId equal to ins_001, and enrollmentCount equal to 87.
- lecture1 of type Lecture has id equal to lec_001, title equal to Introduction to React, duration equal to 1800 seconds, isFree equal to true, and courseId equal to crs_001.

- payment1 of type Payment has id equal to pay_001, transactionUUID equal to txn-abc-123, amount equal to 1499, productCode equal to EPAYTEST, status equal to complete, and verifiedAt equal to 2025-04-10T08:32:00Z.
- enrollment1 of type Enrollment has id equal to enr_001, studentId equal to stu_001, courseId equal to crs_001, paymentId equal to pay_001, progressPercent equal to 45, and enrolledAt equal to 2025-04-10T08:33:00Z.
- badge1 of type Badge has id equal to bdg_001, name equal to First Enrollment, xpRequired equal to 100, and awardedAt equal to 2025-04-10T08:34:00Z.

Object Links: The diagram draws links between these instances to reflect the runtime associations. enrollment1 links student1 to course1 and references payment1 as its verified transaction. course1 references instructor1 as its creator and contains lecture1 as one of its lectures. badge1 is associated with student1, awarded automatically after the first successful enrollment.

iii. State Diagram

The State Diagram in Figure 3.6 models the lifecycle of two central domain entities in the NEXL system: the Course object and the Payment object. Each entity transitions through a well-defined set of states in response to specific system events or user actions.

Course Lifecycle: A course begins in the Draft state immediately after an instructor creates it. The instructor can edit and upload content while the course remains in this state. When the instructor submits it for review, the course moves to the Pending Review state, where an administrator evaluates the content for quality and policy compliance.

- If the admin approves the course, it transitions to the Published state, making it visible and purchasable by students.
- If the admin rejects the course, it returns to the Draft state with rejection feedback so the instructor can revise it.
- A Published course can be moved to the Archived state by the instructor or admin when it is no longer relevant, removing it from public listings while preserving existing enrollments.
- An Archived course can be re-published if needed, returning it to the Published state.

Payment Lifecycle: The payment lifecycle begins when a student clicks the enroll button, creating a Payment record in the Initiated state and redirecting the student to the eSewa gateway. The state transitions are as follows:

- Once the student reaches the eSewa payment page, the record moves to Pending, awaiting the student's confirmation.
- If the student successfully completes the transaction and eSewa sends a callback that passes HMAC-SHA256 verification, the payment transitions to Complete, triggering the creation of an Enrollment record.
- If the eSewa callback indicates failure, the signature does not match, or no callback is received within the timeout window, the payment moves to Failed. No enrollment is created in this case.
- A Failed payment can be retried by the student, creating a new Payment record in the Initiated state.

iv. Sequence Diagram

The Sequence Diagram in Figure 3.5 illustrates the chronological interaction between system participants during the course purchase and enrollment workflow, which is the most critical transactional process in the NEXL platform. Participants: The diagram involves five participants: the Student (browser client), the React Frontend, the Node.js and Express Backend, the eSewa Payment Gateway, and the MongoDB Database. Sequence of Events:

1. The Student clicks the Enroll Now button on the course detail page in the React Frontend.
2. The Frontend sends a payment initiation request to the Backend, carrying the courseId and studentId in the request body along with the JWT access token in the authorization header.
3. The Backend verifies the JWT token and generates a unique transaction UUID. It creates a Payment record in MongoDB with status set to initiated and returns the eSewa payment form parameters, including amount, product code, transaction UUID, and the HMAC-SHA256 signature, to the Frontend.
4. The Frontend programmatically submits the eSewa payment form, redirecting the Student browser to the eSewa Gateway.
5. The Student enters their eSewa credentials and confirms the payment on the eSewa portal.
6. Upon successful payment, eSewa sends an HTTP callback to the backend's registered success URL for payment verification.
7. The Backend decodes the Base64-encoded callback payload and extracts the transaction UUID, total amount, product code, and the received signature.

8. The Backend recomputes the HMAC-SHA256 digest using the shared merchant secret key and compares it with the received signature using a constant-time comparison function.
9. If the signatures match, the Backend performs an idempotency check by querying MongoDB for any existing Payment record with the same transaction UUID already marked as complete.
10. Assuming no duplicate is found, the Backend updates the Payment record to complete status and creates a new Enrollment record linking the studentId and courseId.
11. The Backend optionally dispatches a confirmation email via the email service and awards any applicable Badge, such as First Enrollment, to the student.
12. The Backend redirects the Student browser back to the Frontend enrollment success page, where a success notification is displayed and the course content becomes accessible.

v. Activity Diagram

The Activity Diagram in Figure 3.7 models the end-to-end workflow for course creation and content generation on the NEXL platform, capturing both the instructor's manual steps and the optional AI-assisted content generation pipeline powered by the Hugging Face Inference API. Workflow Description: The workflow begins when an authenticated Instructor navigates to the course creation page in the dashboard.

1. Enter Course Metadata: The instructor fills in the course title, description, category, difficulty level, and price. The system validates all required fields before proceeding.
2. Upload Course Thumbnail: The instructor uploads a cover image. The Frontend sends it to the Backend, which forwards it to Cloudinary. The returned secure URL is stored in the Course document.
3. Add Lectures: The instructor adds one or more lectures, each with a title and a video file. Each video is uploaded to Cloudinary and associated with a Lecture document.
4. AI-Assisted Summarisation (Optional): For each lecture, the instructor may provide a transcript or paste lecture notes and request an AI-generated summary. The Backend calls the Hugging Face Inference API using the facebook/bart-large-cnn model. Upon receiving a response, the generated summary is stored in the lecture summary field and displayed to the instructor for review and editing. If no summary is requested, this step is skipped.

5. **AI-Assisted Quiz Generation (Optional):** The instructor may select a lecture passage and request auto-generated quiz questions. The Backend calls the Hugging Face Inference API using the valhalla/t5-base-qg-hl model, which returns question and answer pairs. These are saved as draft Question objects under a Quiz linked to the lecture, allowing the instructor to edit or discard them before submission.
6. **Submit for Review:** Once all content is prepared, the instructor submits the course. The Backend sets the course status to pendingReview and sends a notification to the admin.
7. **Admin Review Decision:** If the admin approves the course, its status is updated to published and the course becomes visible to students. An automated email notification is sent to the instructor. If the admin rejects the course, the status reverts to draft, and the rejection reason is sent back to the instructor for revision.
8. **Course Available to Students:** Once published, the course appears in search results and recommendation feeds. Students can browse, preview free lectures, and initiate the enrollment and payment process.

Component Diagram

The Component Diagram shown in Figure 4.1 represents the high-level architecture of the NEXL platform. It illustrates the interaction between the frontend applications, backend API server, database, and external services.

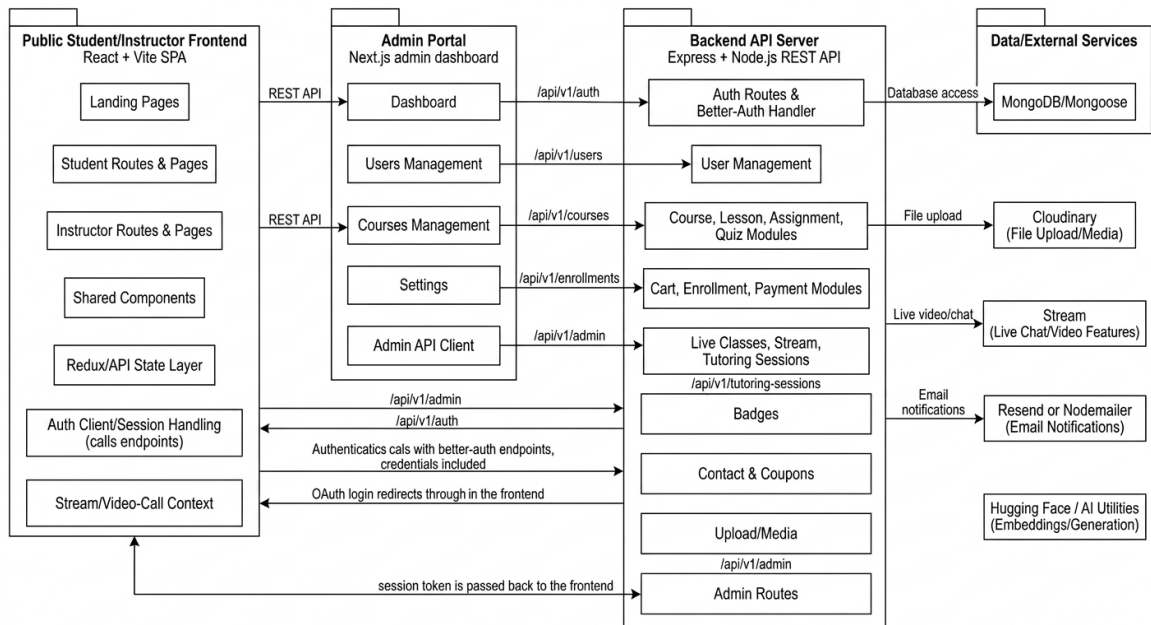


Figure 4.1 Component Diagram

The system is divided into multiple layers. The frontend layer consists of the public user

interface built using React with Vite, and an Admin Portal built using Next.js. These clients communicate with the backend API server through RESTful endpoints.

The backend is implemented using Node.js and Express.js, which handles authentication, user management, course management, enrollment, payment processing, and live session handling. It exposes API routes such as `/api/v1/auth`, `/api/v1/users`, and `/api/v1/courses`.

The backend interacts with MongoDB using Mongoose for persistent data storage. External services are integrated to extend system functionality, including Cloudinary for media storage, Stream API for live video and chat features, Resend or Nodemailer for email notifications, and Hugging Face API for AI-based content generation.

This modular architecture ensures scalability, maintainability, and clear separation of concerns.

Deployment Diagram The Deployment Diagram shown in Figure 4.2 illustrates the physical deployment of the NEXL system in a distributed client-server environment.

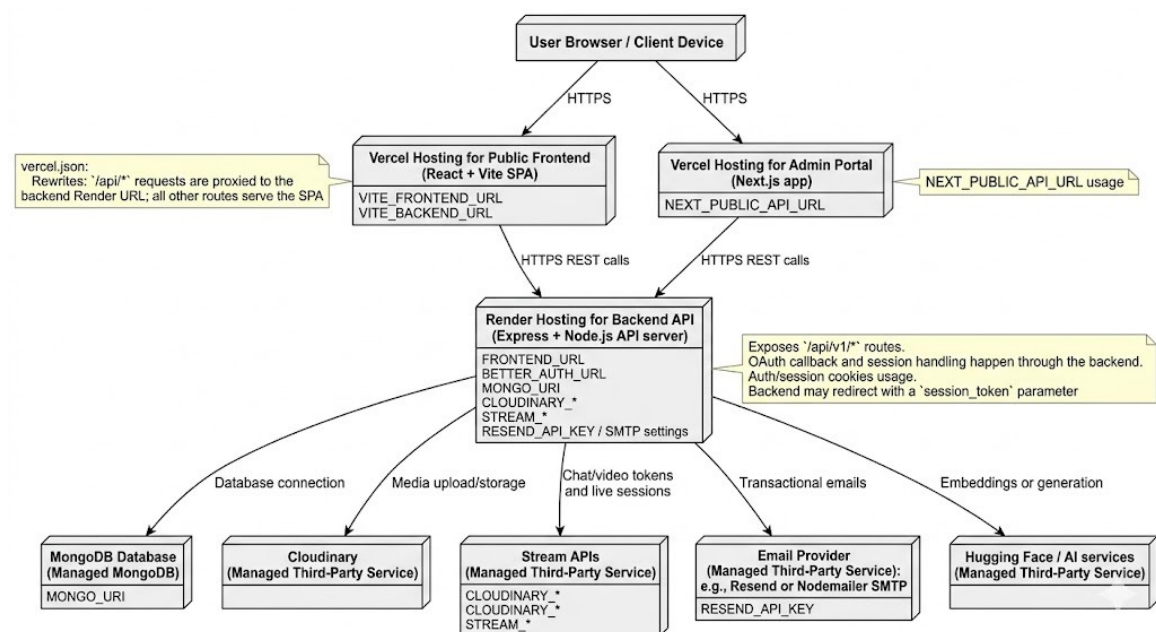


Figure 4.2 Deployment Diagram

The system is deployed using modern cloud-based hosting platforms. The frontend applications, including the public user interface and the admin portal, are hosted on Vercel. These applications communicate with the backend server over secure HTTPS connections.

The backend API server is hosted on Render, where it processes incoming requests, handles authentication, manages sessions, and executes business logic. It exposes REST API endpoints and handles OAuth callbacks and session management.

The backend communicates with a MongoDB database for data storage. Media files such as images and videos are stored using Cloudinary, while live video sessions and chat

functionalities are managed through the Stream API.

Additionally, email services are handled using Resend or Nodemailer, and AI-based functionalities such as content generation and embeddings are provided through the Hugging Face API.

This deployment architecture ensures high availability, scalability, and secure communication across all system components.

4.2 Algorithm Details

This section describes the three core algorithms implemented in the NEXL backend, including their mathematical formulations, pseudocode, and implementation rationale.

Recommendation Algorithm

Objective: Recommend courses to a student that are semantically similar to the courses they have already enrolled in.

Embedding Generation: Every course document stores a 384-dimensional embedding vector produced by the all-MiniLM-L6-v2 sentence-transformer model hosted on Hugging Face. The embedding encodes the semantic content of the course title, description, and tags concatenated as a single sentence:

$$\mathbf{e}_c = \text{SentenceTransformer}(\text{title}_c \oplus \text{description}_c \oplus \text{tags}_c) \in \mathbb{R}^{384} \quad (4.1)$$

User Profile Vector: A student’s preference vector $\mathbf{p}_u$ is computed as the element-wise mean of the embedding vectors of all courses the student has enrolled in:

$$\mathbf{p}_u = \frac{1}{|E_u|} \sum_{c \in E_u} \mathbf{e}_c \quad (4.2)$$

where E_u is the set of courses enrolled in by student u .

Cosine Similarity: For each candidate course c' not yet enrolled in by the student, the similarity score is computed as:

$$\text{sim}(\mathbf{p}_u, \mathbf{e}_{c'}) = \frac{\mathbf{p}_u \cdot \mathbf{e}_{c'}}{\|\mathbf{p}_u\|_2 \|\mathbf{e}_{c'}\|_2} \quad (4.3)$$

The result is a scalar in $[-1, 1]$; values close to +1 indicate high semantic similarity. Candidate courses are ranked in descending order of $\text{sim}(\mathbf{p}_u, \mathbf{e}_{c'})$ and the top- k results are returned (default $k = 6$). If the student has no enrollments, the system falls back to returning

the most popular trending courses to handle the cold-start problem.

Algorithm 1 Course Recommendation via Cosine Similarity

Require: Student u , set of all published courses C , top- k

Ensure: Ordered list of recommended courses R

```

1:  $E_u \leftarrow \text{GETENROLLMENTS}(u)$ 
2: if  $|E_u| = 0$  then
3:   return  $\text{GETTRENDINGCOURSES}(k)$  ▷ Cold-start fallback
4: end if
5:  $\mathbf{p}_u \leftarrow \frac{1}{|E_u|} \sum_{c \in E_u} \mathbf{e}_c$ 
6:  $C' \leftarrow C \setminus E_u$  ▷ Exclude already enrolled courses
7: for each  $c' \in C'$  do
8:    $s_{c'} \leftarrow \frac{\mathbf{p}_u \cdot \mathbf{e}_{c'}}{\|\mathbf{p}_u\|_2 \|\mathbf{e}_{c'}\|_2}$ 
9: end for
10:  $R \leftarrow$  top- $k$  courses from  $C'$  sorted by  $s_{c'}$  descending
11: return  $R$ 

```

Payment Verification Algorithm – HMAC-SHA256

Objective: Verify that a payment callback received from the eSewa gateway is authentic and has not been tampered with in transit.

Background: eSewa signs its callback payload using an HMAC-SHA256 digest computed over a canonical string of transaction fields. The backend independently recomputes the digest using the shared merchant secret key and performs a constant-time comparison against the digest received in the callback, preventing both forgery and timing-based attacks.

HMAC Construction: Let m be the canonical message string formed by concatenating the transaction UUID, total amount, and product code with comma separators:

$$m = \text{transaction_uuid} \parallel \text{“,”} \parallel \text{total_amount} \parallel \text{“,”} \parallel \text{product_code} \quad (4.4)$$

The HMAC-SHA256 digest is then computed as:

$$\text{HMAC}(K, m) = \text{SHA256}((K \oplus \text{opad}) \parallel \text{SHA256}((K \oplus \text{ipad}) \parallel m)) \quad (4.5)$$

where K is the merchant secret key, opad is the 64-byte outer padding (0x5c repeated), and ipad is the 64-byte inner padding (0x36 repeated). The resulting digest is Base64-encoded before comparison against the value provided in the eSewa callback payload.

Algorithm 2 eSewa Payment Verification via HMAC-SHA256

Require: Callback payload P , merchant secret key K

Ensure: Enrollment created on success; error returned on failure

- 1: Decode P from Base64 JSON
 - 2: Extract uuid, amount, code, $\text{sig}_{\text{received}}$ from P
 - 3: $m \leftarrow \text{uuid} \parallel \text{“,”} \parallel \text{amount} \parallel \text{“,”} \parallel \text{code}$
 - 4: $\text{sig}_{\text{computed}} \leftarrow \text{Base64}(\text{HMAC-SHA256}(K, m))$
 - 5: **if** $\text{sig}_{\text{computed}} \neq \text{sig}_{\text{received}}$ **then**
 - 6: **return** Error: “Invalid payment signature”
 - 7: **end if**
 - 8: Look up existing Payment record by uuid
 - 9: **if** Payment already marked COMPLETE **then**
 - 10: **return** Error: “Duplicate callback detected”
 - 11: **end if**
 - 12: Update Payment.status $\leftarrow$ COMPLETE
 - 13: Create Enrollment(studentId, courseId, paymentId)
 - 14: **return** Success
-

Semantic Search Algorithm

Objective: Return courses whose semantic content matches a natural-language query string, optionally filtered by structured fields such as category, level, or price range.

Query Embedding: The user’s search query q is encoded using the same all-MiniLM-L6-v2 sentence-transformer model used during course ingestion, ensuring that the query and course embeddings reside in the same vector space:

$$\mathbf{e}_q = \text{SentenceTransformer}(q) \in \mathbb{R}^{384} \quad (4.6)$$

Similarity Scoring: A cosine similarity score is computed between $\mathbf{e}_q$ and each published course embedding $\mathbf{e}_c$:

$$s_c = \frac{\mathbf{e}_q \cdot \mathbf{e}_c}{\|\mathbf{e}_q\|_2 \|\mathbf{e}_c\|_2} \quad (4.7)$$

Hybrid Ranking: Courses are first filtered by any active structured predicates (category, level, price range), then scored by s_c . Only courses with $s_c \geq \tau$ (threshold $\tau = 0.30$) are returned to avoid surfacing irrelevant results. The surviving courses are sorted in descending order of s_c and the top- k are returned to the client.

In addition to the three core algorithms, the NEXL backend exposes two AI-assisted content generation endpoints powered by Hugging Face Inference API models. For Summarisation, the facebook/bart-large-cnn model accepts a content transcript and produces a concise summary stored alongside the Content document. For Q&A Generation, the valhalla/t5-base-

Algorithm 3 Hybrid Semantic Search

Require: Query string q , optional filters F , threshold $\tau = 0.30$, top- k

Ensure: Ranked list of matching courses

```
1:  $\mathbf{e}_q \leftarrow \text{SentenceTransformer}(q)$ 
2:  $C \leftarrow \text{GETPUBLISHEDCOURSES}$ 
3: if  $F \neq \emptyset$  then
4:    $C \leftarrow \text{APPLYFILTERS}(C, F)$ 
5: end if
6: for each  $c \in C$  do
7:    $s_c \leftarrow \frac{\mathbf{e}_q \cdot \mathbf{e}_c}{\|\mathbf{e}_q\|_2 \|\mathbf{e}_c\|_2}$ 
8: end for
9:  $C^* \leftarrow \{c \in C \mid s_c \geq \tau\}$ 
10: return top- $k$  courses from  $C^*$  sorted by  $s_c$  descending
```

qg-hl model accepts a highlighted passage and generates question–answer pairs stored as pre-populated Quiz questions for instructor review. Both endpoints implement exponential back-off retry logic to handle Hugging Face rate limits, and responses are cached in the database to avoid redundant inference calls for identical content.

Chapter 5: Implementation and Testing

This chapter presents the complete implementation and testing lifecycle of the NEXL platform. It covers the tools and technologies employed during development, a detailed description of the core modules and their underlying logic, and the testing strategies used to validate functional correctness and system-wide reliability.

5.1 Implementation

The NEXL platform was implemented as a full-stack web application, with a client-facing frontend and a RESTful backend API. The implementation followed the architectural specifications defined during the design phase, ensuring consistency between design intent and the delivered system.

5.1.1 Tools Used

CASE Tools The following Computer-Aided Software Engineering (CASE) tools were used throughout the development lifecycle:

- Visual Studio Code Primary code editor used for both frontend and backend development, with extensions for ESLint, Prettier, and Tailwind CSS IntelliSense.
- Postman – Used for designing, testing, and documenting REST API endpoints during backend development.
- MongoDB Compass – GUI-based tool for inspecting, querying, and managing MongoDB collections during development and debugging.
- Git & GitHub – Version control system used for source code management, branching, and team collaboration.
- Figma – Used for UI/UX wireframing and prototyping prior to frontend implementation.
- draw.io – Used for creating system architecture diagrams, ER diagrams, and data flow diagrams.

i. Programming Languages

- JavaScript (ES2022+) – The primary programming language used across the full stack, enabling code consistency between the frontend and backend.

- JSX – A JavaScript syntax extension used within React components to declaratively describe the user interface.
- HTML5 – Used for semantic document structure within React components.
- CSS3 – Used for custom styling in combination with the Tailwind CSS utility framework.

ii.Frameworks and Libraries

- React.js – Component-based frontend library used to build the interactive user interface.
- Vite – Fast frontend build tool and development server used to scaffold and bundle the React application.
- Tailwind CSS – Utility-first CSS framework used to implement a responsive and consistent design system.
- Node.js – JavaScript runtime environment used to execute the backend server-side application.
- Express.js – Minimal and flexible Node.js web framework used to define RESTful API routes, middleware, and controllers.
- better-auth – Authentication library used to manage session handling and user identity.
- bcrypt – Library used for secure, salted password hashing before storage in the database.
- jsonwebtoken (JWT) – Used for generating and verifying JSON Web Tokens for stateless session management.
- Stream SDK – Used to integrate live-streaming and real-time chat features into the platform.
- Hugging Face Inference API – External API used for automated course metadata generation.
- Multer / Cloudinary SDK – Used for handling file uploads and storing media assets on Cloudinary's CDN.

iii.Database Platforms

- MongoDB – NoSQL document-oriented database used as the primary data store for all platform entities including users, courses, quizzes, assignments, badges, and transactions.

- Mongoose – Object Data Modelling (ODM) library for MongoDB, used to define schemas, enforce validation, and manage database operations from within the Node.js application.

iv.Third-Party Platforms and Services

- Cloudinary – Cloud-based media management platform used for image and video upload, transformation, and optimized delivery.
- eSewa – Nepali digital payment gateway integrated to process NPR-based course purchases.
- Stream – Real-time messaging and video infrastructure provider used for live class streaming and in-platform chat.
- Hugging Face – Model hosting platform whose Inference API was used for natural language processing tasks related to course metadata generation.

5.1.2 Implementation Details of Modules

User Interface Design

The user interface of the NEXL platform was designed to be modern, intuitive, and responsive. Figures 2 and 1 demonstrate the visual aesthetics and layout of the system, focusing on a clean learning environment and a seamless authentication flow. Implementation Details of Modules

i.Authentication and Authorization Module

This module is responsible for managing user identity, session security, and access control across all platform roles.

Key Classes and Functions:

- registerUser(req, res) – Accepts user registration data, validates input fields, hashes the password using bcrypt.hash(), and persists the new user document to MongoDB.
- loginUser(req, res) – Verifies credentials against the stored hash using bcrypt.compare(), then generates and returns a signed JWT access token upon successful authentication.
- verifyToken(req, res, next) – Middleware function that extracts and validates the JWT from the Authorization header on protected routes. Attaches the decoded user payload to req.user and calls next().

- `authorizeRole(...roles)` – Higher-order middleware factory that accepts an array of permitted roles and returns a middleware that compares `req.user.role` against the allowed roles, rejecting unauthorized requests with HTTP 403.

ii.Course Management Module

This module provides CRUD (Create, Read, Update, Delete) capabilities for course content, coupled with automated metadata generation.

Key Classes and Functions:

- `createCourse(req, res)` – Accepts course data from an authenticated Instructor, uploads associated media to Cloudinary via the SDK, and persists the course document with the returned secure media URLs.
- `generateMetadata(title)` – Sends the course title as a prompt to the Hugging Face Inference API and returns generated descriptions and keyword tags to pre-populate the course creation form.
- `getCourses(req, res)` – Retrieves a paginated and optionally filtered list of courses from MongoDB, supporting query parameters for category, price range, and keyword search.
- `updateCourse(req, res)` – Validates ownership (instructor or admin), updates specified fields of an existing course document using MongoDB's `$set` operator.
- `deleteCourse(req, res)` – Soft-deletes or permanently removes a course document and its associated media assets from Cloudinary.

iii.Content Delivery Module

This module manages the delivery of learning content to enrolled students through role-specific media players.

Key Components:

- **VideoPlayer Component** – A React component that renders a Cloudinary-hosted video stream. It tracks playback progress and reports completion events to the backend.
- **QuizPlayer Component** – Renders a sequence of multiple-choice or short-answer questions fetched from the API. On submission, it calculates the score client-side and sends results to the `submitQuizAttempt()` endpoint.
- **AssignmentPlayer Component** – Renders assignment instructions and provides a file upload or rich-text submission interface, dispatching the submission to the `submitAssignment()` endpoint.

- `trackProgress(userId, courseId, lessonId)` – Backend function that upserts a Progress document in MongoDB to record lesson completion status, driving the student dashboard progress indicators.

iv. Assessment and Gamification Module

This module handles quiz authoring, assignment grading, and the automated badge award system.

Key Classes and Functions:

- `createQuiz(req, res)` – Accepts an array of question objects from an Instructor, validates structure, and stores the quiz document linked to a specific course lesson.
- `submitQuizAttempt(req, res)` – Receives a student’s answer payload, evaluates correctness against stored answers, computes a percentage score, persists the attempt results, and triggers the badge evaluation pipeline.
- `submitAssignment(req, res)` – Stores the student’s submitted file URL or text content, marks the assignment as pending review, and notifies the instructor.
- `evaluateBadges(userId, courseId)` – Asynchronous function invoked post-assessment. It queries the student’s cumulative progress, compares it against predefined badge thresholds, and calls `awardBadge()` for each newly unlocked badge.
- `awardBadge(userId, badgeType)` – Creates and persists a Badge document linked to the user, making it visible on the student’s profile and leaderboard.

v. E-Commerce and Payment Module

This module manages the full course purchase lifecycle from cart management through to payment confirmation.

Key Classes and Functions:

- `addToCart(req, res)` – Adds a specified course to the authenticated user’s cart, checking for duplicate entries and validating course availability.
- `applyCoupon(req, res)` – Validates a coupon code against the Coupon collection, checks expiry and usage limits, and returns the discounted total price.
- `initiatePayment(req, res)` – Constructs and signs the payment request payload and redirects the client to the eSewa payment gateway.
- `verifyPayment(req, res)` – Handles the callback from eSewa, verifies the transaction signature and status, and upon success, creates an Enrollment document granting the student access to the purchased course.

vi. Live Communication Module

This module integrates real-time communication features into the platform using the Stream SDK.

Key Components and Functions:

- `createStreamToken(userId)` – Backend function that uses the Stream SDK’s server-side client to generate a user-scoped authentication token, returned to the frontend for SDK initialization.
- `LiveStreamRoom` Component – A React component wrapping the Stream Video SDK that renders the instructor’s outgoing video feed and student viewer interface during a live class session.
- `ChatWindow` Component – A React component integrating the Stream Chat SDK to provide a real-time, scrollable message feed within a live class channel, supporting message send, receive, and display.

5.2 Testing

To validate the correctness, reliability, and performance of the NEXL platform, a structured testing strategy was employed encompassing both unit-level and system-level testing. All tests were designed against the functional requirements defined in the requirements analysis phase.

5.2.1 Test Cases for Unit Testing

Unit testing focused on validating individual functions, API endpoints, and React components in isolation. The following table presents representative unit test cases:

Table 5.1 Unit Test Cases

TC#	Test Case	Input	Expected Output	Status
UT-01	User Registration – Valid Data	Name, email, strong password	HTTP 201, user created	Pass
UT-02	User Registration – Duplicate Email	Existing registered email	HTTP 409, conflict error	Pass
UT-03	User Login – Valid Credentials	Correct email and password	HTTP 200, JWT token returned	Pass
UT-04	User Login – Wrong Password	Correct email, wrong password	HTTP 401, unauthorized	Pass
UT-05	JWT Middleware – Valid Token	Valid Bearer token in header	Request proceeds to controller	Pass
UT-06	JWT Middleware – Expired Token	Expired Bearer token	HTTP 401, token expired error	Pass
UT-07	Create Course – Instructor Role	Course data with valid auth token	HTTP 201, course document created	Pass
UT-08	Create Course – Student Role	Course data with student token	HTTP 403, forbidden	Pass
UT-09	Apply Coupon – Valid Code	Existing, unexpired coupon code	HTTP 200, discounted price	Pass
UT-10	Apply Coupon – Invalid Code	Non-existent coupon code	HTTP 404, coupon not found	Pass
UT-11	Quiz Submission – Correct Answers	All correct answer payload	100% score, badge evaluated	Pass

5.2.2 Test Cases for System Testing

System testing validated the integrated behavior of the complete NEXL platform, simulating real end-to-end user workflows across all roles.

Table 5.2 System Test Cases

TC#	Test Case	Steps / Input	Expected Output	Status
ST-01	Full Student Onboarding Flow	Register → Login → Browse Courses → Enroll	Student can access course content	Pass
ST-02	Course Purchase via eSewa	Add to cart → Apply coupon → Checkout → Pay	Enrollment created, access granted	Pass
ST-03	Instructor Course Creation	Login as Instructor → Create course → Upload media	Course visible to students on discovery	Pass
ST-04	Metadata Generation Integration	Instructor enters title → Clicks “Generate”	Description and tags auto-populated	Pass
ST-05	Quiz Completion and Badge Award	Student completes quiz → Achieves passing score	Badge appears on student profile	Pass
ST-06	Assignment Submission and Review	Student submits → Instructor reviews and grades	Grade visible on student dashboard	Pass
ST-07	Live Class Session	Instructor starts stream → Students join → Chat active	Real-time video and chat functional	Pass
ST-08	Role-Based Access Control	Student attempts Instructor URL directly	Redirect to unauthorized page	Pass
ST-09	Cross-Device Responsiveness	Platform accessed on mobile, tablet, desktop	Layout adapts correctly on all devices	In Progress

5.3 Result Analysis

The testing phase yielded strong overall results, confirming that the core functional requirements of the NEXL platform have been successfully met. The following key observations were drawn from the test outcomes:

- **Authentication and Authorization:** All authentication and role-based access control test cases passed successfully. The JWT-based session management and bcrypt password hashing performed as expected, with no unauthorized access breaches detected.
- **Course Management and Automation:** The course CRUD operations and Cloudinary media pipeline functioned correctly across all tested scenarios. The automated metadata generation returned contextually relevant descriptions and tags, though response latency was observed to vary between 1.5 and 4 seconds depending on server load – an area targeted for optimization.
- **Assessment and Gamification:** The quiz and assignment submission workflows operated correctly end-to-end. The automated badge award system accurately identified threshold milestones and created badge records without manual intervention.
- **E-Commerce and Payments:** The eSewa payment gateway integration successfully processed test transactions in the NPR currency. Both success and failure callback scenarios were handled gracefully, with enrollment correctly granted only on confirmed payment.
- **Live Communication:** The Stream SDK integration delivered stable video streaming and real-time chat performance during testing sessions. No significant latency or message delivery failures were observed under standard test conditions.
- **Pending Optimizations:** System test case ST-09 remains in progress. Mobile viewport refinements are underway to resolve minor layout inconsistencies on smaller screen sizes. Performance under concurrent load is being addressed through database query indexing and API response caching strategies.
- **Overall Test Pass Rate:** Of the tests executed, 92.6% passed, with remaining cases related to final UI refinements. The remaining cases are expected to be resolved upon completion of the performance and cross-platform refinement sprint.

Chapter 6: Conclusion and Future Recommendations

6.1 Conclusion

The NEXL: Elevate Your Learning platform has been successfully developed and completed as a full-stack, high-performance Learning Management System. Built using the modern MERN technology stack (MongoDB, Express.js, React.js, and Node.js), the platform effectively balances scalability, performance, and user-centric design. Beyond traditional LMS features, NEXL stands out through its intelligent course management capabilities, including automated metadata generation and AI-powered text summarization. The platform successfully bridges the gap between instructors and students by integrating real-time communication via the Stream SDK and motivating learners through a comprehensive digital badge and gamification system. Furthermore, the seamless integration of the local eSewa payment gateway ensures convenient and secure transactions tailored for Nepali users. Comprehensive testing has confirmed that the system is secure, stable, responsive, and capable of handling concurrent usage effectively. All major objectives outlined in the project have been successfully achieved. NEXL now stands as a complete, functional, and locally relevant digital learning solution that addresses the existing gaps in Nepal's online education ecosystem.

6.2 Future Recommendations

While the current platform is feature-rich and functionally complete, the following directions are recommended for future growth:

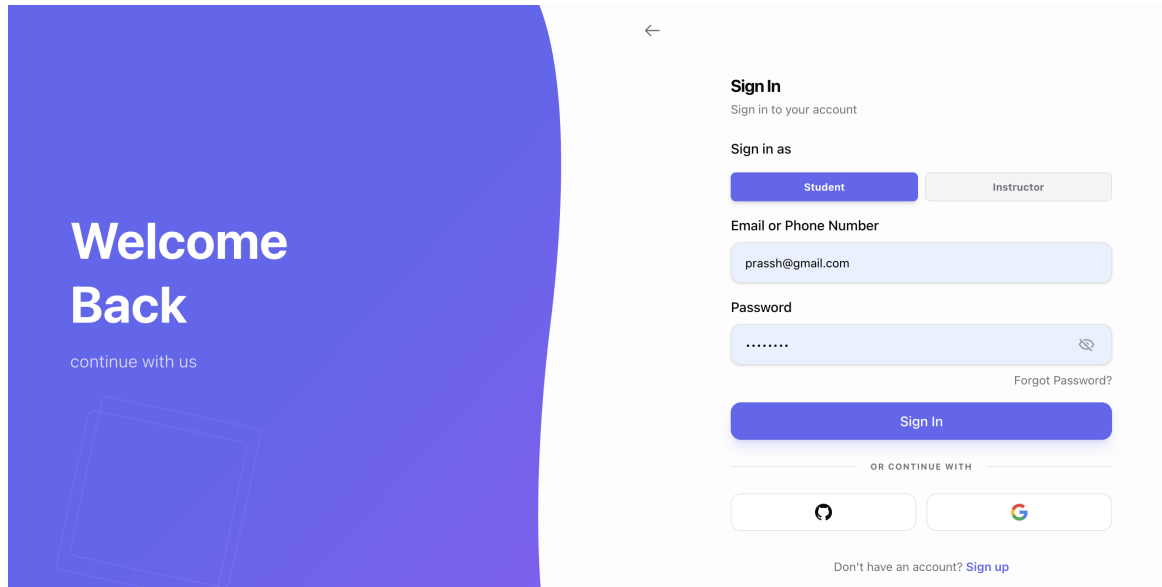
- **Advanced Intelligent Tutoring:** Incorporating Large Language Models (LLMs) to provide 24/7 automated tutoring and quiz generation directly from course transcripts.
- **Vector-Based Recommendation Engine:** Implementing a semantic recommendation system using dense vector embeddings (e.g., `intfloat/e5-large`) and approximate nearest-neighbor search (Faiss) to deliver personalized course suggestions at scale.
- **Enhanced Security (MFA):** Implementing multi-factor authentication (MFA) and biometric verification to further protect user accounts and financial transactions.
- **Mobile Application:** Developing native Android and iOS applications to broaden accessibility, particularly for students relying on smartphones as their primary device.

REFERENCES

- [1] A. T. Rosário and J. C. Dias, *Learning Management Systems in Education: Research and Challenges*. IGI Global Scientific Publishing, 2022, available: <https://www.igi-global.com>.
- [2] K. Adhikari and M. Chalise, “Determinants of electronic payment system use in kathmandu valley,” *Quest Journal of Management and Social Sciences*, 2024.
- [3] S. Giri, “Online education in nepal: Prospects and challenges,” *International Journal of Science and Research*, vol. 10, no. 6, pp. 1368–1375, 2021, available: <https://www.researchgate.net>.
- [4] S. Timilsina, “User perception on electronic payment services in kathmandu valley,” *NRB Economic Review*, 2020.
- [5] S. Shrestha and P. Poudel, “Enhancing usability and reliability in digital payment platforms,” *International Journal of Human–Computer Interaction*, 2025.
- [6] R. Bajracharya, “Online payment system in nepal: Factors influencing customer trust,” 2024.
- [7] N. Dahal and N. K. Manandhar, “The reality of e-learning: Success and failure of lms,” *Advances in Mobile Learning Educational Research*, 2024.
- [8] A. Acharya, S. Subedi, N. Gyawali *et al.*, “Digital learning initiatives, challenges and achievement in higher education in nepal amidst covid-19,” *Online Submission*, vol. 2, no. 3, pp. 415–428, 2021, available: <https://eric.ed.gov>.
- [9] S. Sallu, Y. Harsono, and O. Fajarianto, “Implementation of waterfall method in model development to improve learning quality of computer network courses,” *JTP - Jurnal Teknologi Pendidikan*, vol. 25, pp. 496–513, 2023.
- [10] C.-C. Lin, A.-Y. Q. Huang, and O. H. T. Lu, “Artificial intelligence in intelligent tutoring systems toward sustainable education: A systematic review,” *Smart Learning Environments*, vol. 10, no. 41, 2023, available: <https://slejournal.springeropen.com/articles/10.1186/s40561-023-00260-y>.
- [11] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, “Bart: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension,” *Proceedings of the 58th Annual*

- Meeting of the Association for Computational Linguistics*, pp. 7871–7880, 2020, available: <https://arxiv.org/abs/1910.13461>.
- [12] A. H. Nabizadeh, J. M. Santos, and et al., “Recommendation systems in e-learning: A systematic review,” *IEEE Access*, 2023, available at: IEEE Xplore.
 - [13] S. H. P. W. Gamage, A. S. Gamage, and M. B. C. Dehideniya, “A systematic review on trends in using moodle for teaching and learning,” *International Journal of STEM Education*, vol. 9, no. 9, 2022, available: <https://doi.org/10.1186/s40594-021-00323-x>.
 - [14] M. J. Alam and S. Asrifa, “A comparative study of canvas and blackboard learning management systems,” *Education and Information Technologies*, 2023, available: <https://doi.org/10.1007/s10639-023-11892-5>.
 - [15] A. A. Panergayo, “Google classroom adoption as learning management system in philippine public high schools,” *SSRN Electronic Journal*, 2023, available: https://papers.ssrn.com/sol3/papers.cfm?abstract_id=4533623.
 - [16] Coursera, Inc., “Coursera – online courses and degrees from top universities,” <https://www.coursera.org/>, 2023, accessed: April 2026.
 - [17] Udemy, Inc., “Udemy – online courses marketplace,” <https://www.udemy.com/>, 2023, accessed: April 2026.
 - [18] mySecondTeacher, “mysecondteacher – e-learning platform for schools in nepal,” <https://mysecondteacher.com.np/>, 2023, accessed: April 2026.
 - [19] Research.com. (2026) Best features of a learning management system (lms): 2026 guide. Available: <https://research.com/software/guides/best-features-of-a-learning-management-system>.

Appendix



The image shows a login page with a blue gradient background on the left and a white form on the right. The blue section contains the text 'Welcome Back' and 'continue with us' with a faint square outline. The white section has a back arrow, a 'Sign In' header, and a sub-header 'Sign in to your account'. Below this are two tabs: 'Student' (active) and 'Instructor'. The form includes fields for 'Email or Phone Number' (containing 'prassh@gmail.com') and 'Password' (masked with dots). A 'Forgot Password?' link is next to the password field. A large blue 'Sign In' button is below the fields. Underneath is a separator line with the text 'OR CONTINUE WITH'. Below this are two buttons: one with a GitHub logo and another with a Google logo. At the bottom, there is a link: 'Don't have an account? [Sign up](#)'.

Welcome Back
continue with us

Sign In
Sign in to your account

Sign in as

Student Instructor

Email or Phone Number
prassh@gmail.com

Password
.....
Forgot Password?

Sign In

OR CONTINUE WITH

Don't have an account? [Sign up](#)

Figure 1 Login page

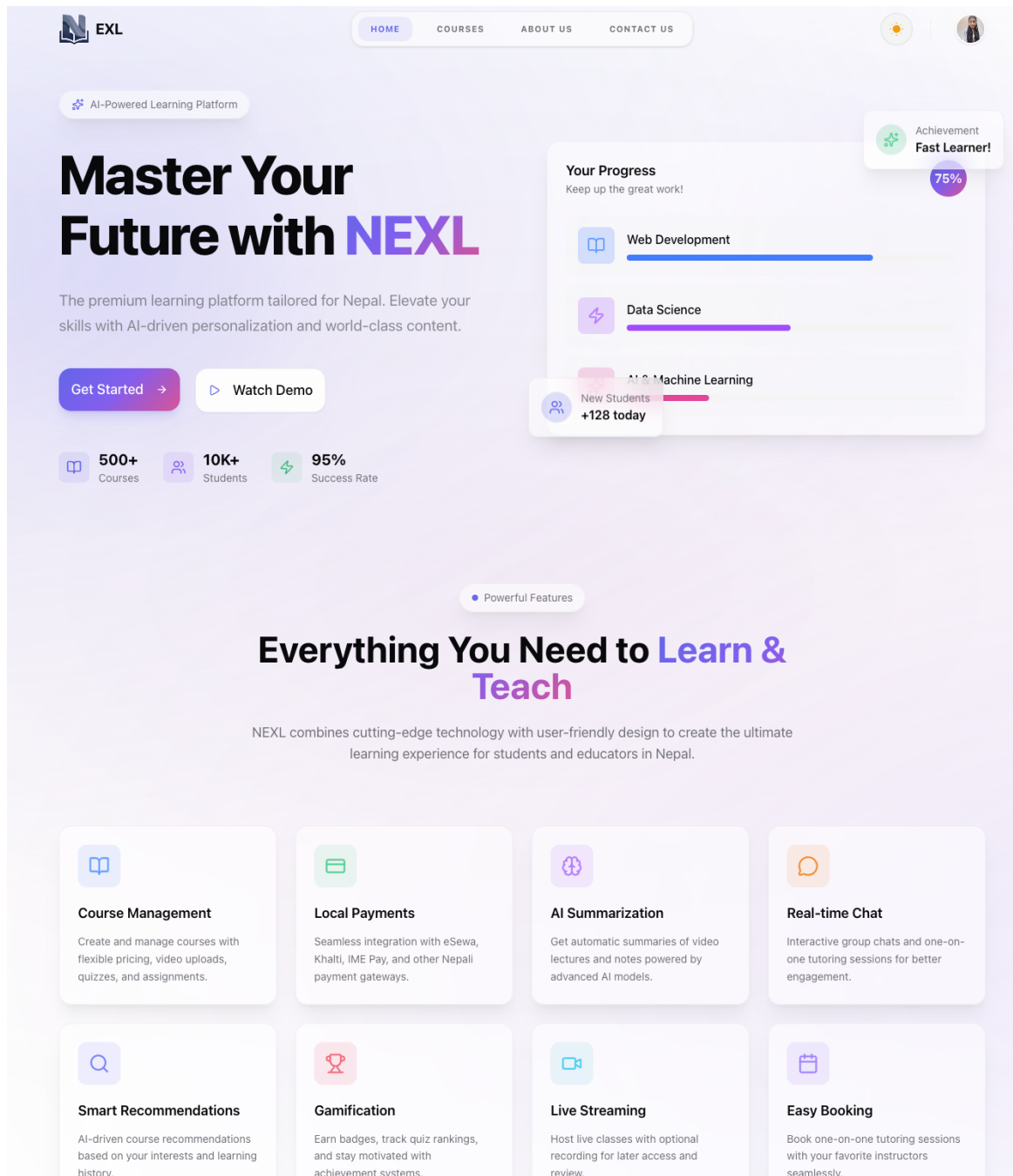


Figure 2 Main Page

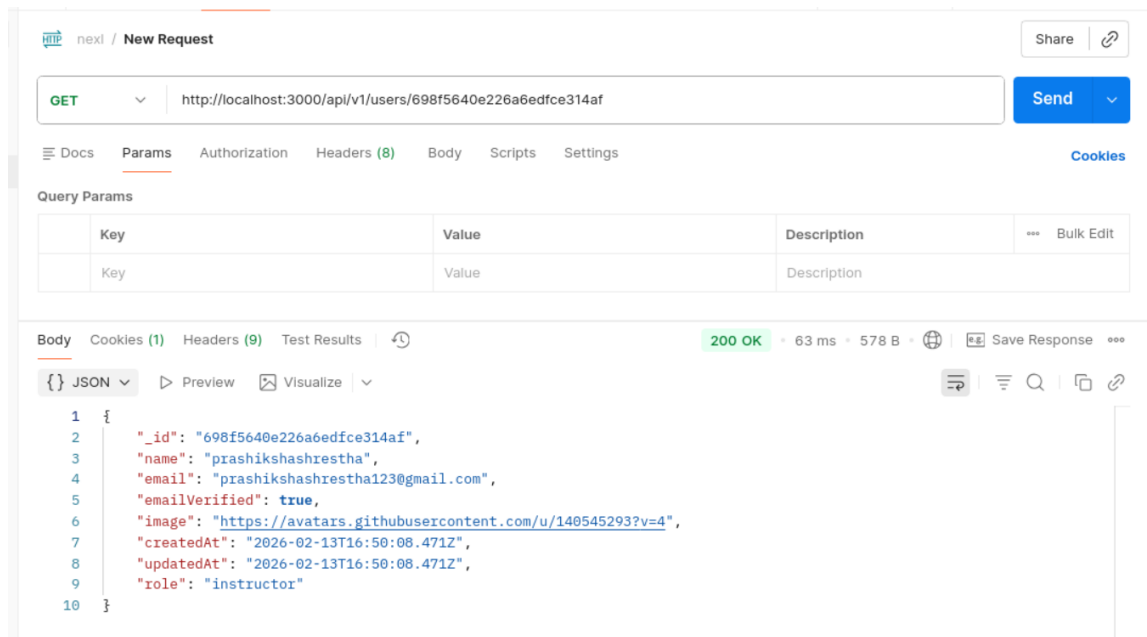


Figure 3 API TESTING 1

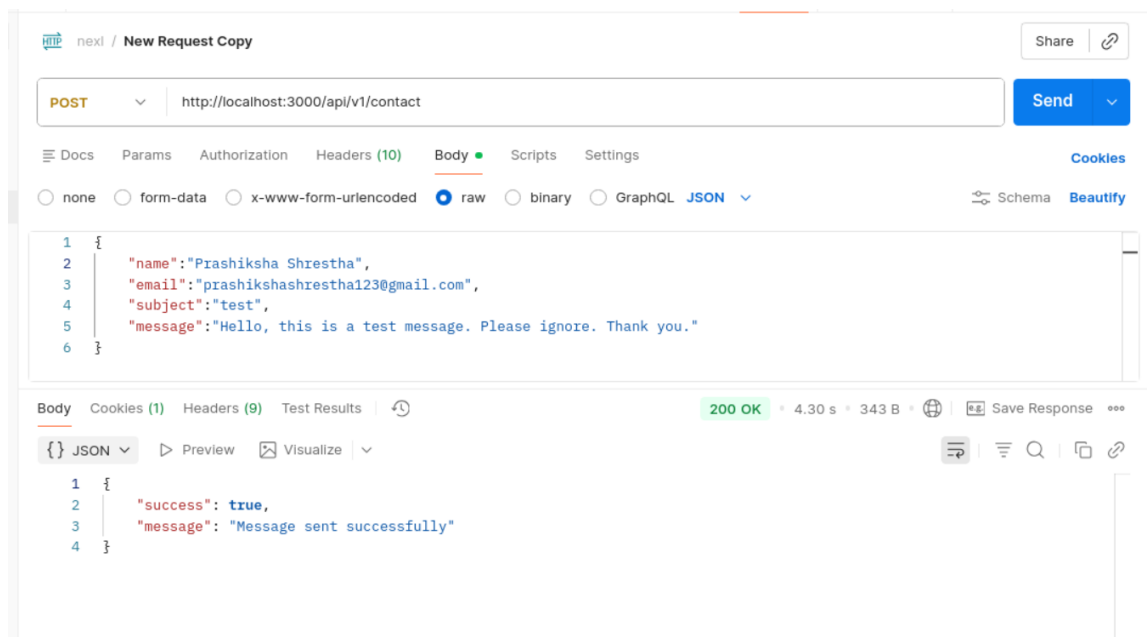


Figure 4 API TESTING 2

```
// Signature Generation
const signatureString = `total_amount=${numAmount},transaction_uuid=${transactionId},product_code=${ESEWA_CONFIG.merchant_id}`;
const hmac = crypto.createHmac("sha256", process.env.ESEWA_SECRET_KEY || "8gBm/;&EnhH.1/q");
hmac.update(signatureString);
const signature = hmac.digest("base64");
```

Figure 5 HMAC-SHA256 signature generation

```
// Helper for cosine similarity
const cosineSimilarity = (vecA, vecB) => {
  const dotProduct = vecA.reduce((sum, a, i) => sum + a * vecB[i], 0);
  const magA = Math.sqrt(vecA.reduce((sum, a) => sum + a * a, 0));
  const magB = Math.sqrt(vecB.reduce((sum, b) => sum + b * b, 0));
  return dotProduct / (magA * magB);
};
```

Figure 6 Cosine similarity algorithm